



算法设计与分析

讲授内容：算法分析

2025年11月14日

- 本章主要知识点：

§ 1.1 算法的基本概念（回顾）

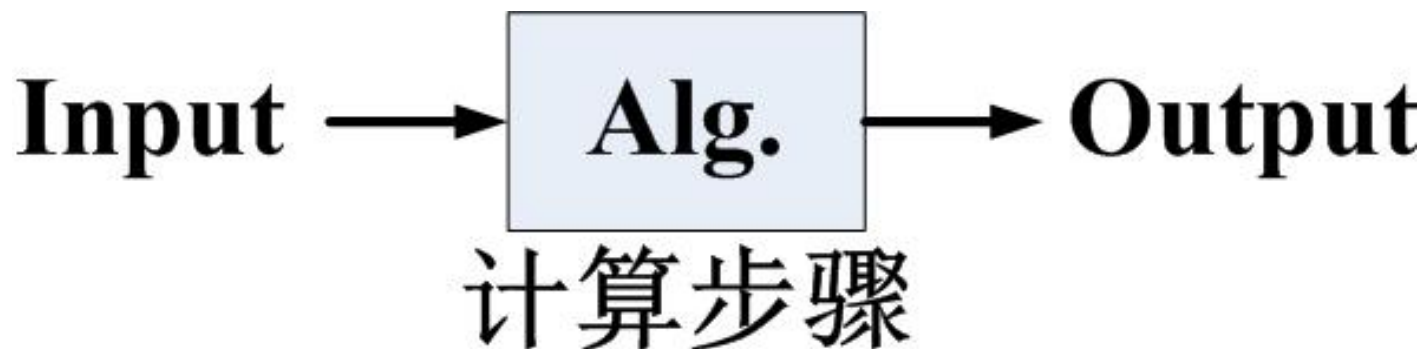
§ 1.2 算法描述（回顾）

§ 1.3 算法分析

§ 1.1 算法的基本概念

- 定义

- 一个算法是任何一个良定义（well-defined）的计算过程，它接收某个值或值的集合作为输入，产生某个值或值的集合作为输出。因此，一个算法是一个计算步骤的序列，这些步骤将输入转化为输出。



§ 1.1 算法的基本概念（续）

- 算法可以看成是用于求解具有良好规格说明的计算问题的工具。
 - 一般来说，问题陈述说明了期望的输入/输出关系
 - 算法则描述一个特定的计算过程来实现该输入/输出关系
- 例：排序问题
 - 问题描述：
 - Input: 具有 n 个数的数列 $\langle a_1, a_2, \dots, a_n \rangle$
 - Output: 上述序列的一个排列 $\langle a_1', a_2', \dots, a_n' \rangle$ 满足关系： $a_1' \leq a_2' \leq \dots \leq a_n'$
 - 计算步骤：
 - 如何达到上述关系

§ 1.1 算法的基本概念（续）

● 问题及实例

— 问题：需要求解的一般性提问，通常含若干参数

— 问题描述：

- 定义问题参数（集合，变量，序列等）
- 说明每个参数的取值范围及参数间的关系
- 定义问题的解
- 说明满足的条件（优化目标或约束条件）

— 问题实例

- 参数的一组赋值可得到问题的一个实例

求解实际问题之前首先要对问题进行建模！

§ 1.1 算法的基本概念（续）

- 基本概念

- **Instance:** 一个问题的实例由计算该问题的一个解所需要的所有输入所组成。

- ✓ 举例（排序问题）：给定输入序列 $\langle 31, 41, 59, 26, 41, 58 \rangle$ ，排序算法将返回序列 $\langle 26, 31, 41, 41, 58, 59 \rangle$ 作为输出。这样的输入序列称为排序问题的一个实例

- 正确性：若对每个输入实例，算法均终止于正确的输出，则称算法是正确的。

- 不正确算法：对某些输入实例不停机，或虽然停机，但不是所期望的答案。

- **Note:** 一个不正确的算法，若其错误的概率是可控制的，有时也是有用的。

§ 1.1 算法的基本概念（续）

- 性质

- 算法是由若干条指令组成的有穷序列，满足以下4条性质：

- 输入：有零个或多个外部量作为算法的输入。
 - 输出：算法产生至少一个量作为输出。
 - 确定性：组成算法的每条指令清晰、无歧义。
 - 有限性：算法中每条指令的执行次数有限，执行每条指令的时间也有限。

- 程序与算法的区别？

- 程序是算法用某种程序设计语言的具体实现，程序可以不满足算法的性质(4)即有限性。

§ 1.1 算法的基本概念（续）

- 程序与算法的区别

- (1) 在语言描述上，程序必须是用规定的程序设计语言来写，而算法较随意（伪代码）；
- (2) 在执行时间上，算法所描述的步骤一定是有限的，而程序可以无限地执行下去。

所以： 程序 = 数据结构 + 算法

- 通俗一些 就是，算法是解决一个问题的思路，程序，则是解决这些问题所具体好写的代码。
- 算法没有语言界限，只是一个思路。为实现相同的一个算法，用不同语言编写的程序会不一样。
- 算法是写程序的思想，程序是算法的体现！

§ 1.1 算法的基本概念（续）

- 算法与数据结构课程的关系
 - 数据结构是底层：存储和组织数据的方式，旨在便于访问和修改
 - 算法是高层：数据结构为算法提供服务；算法围绕数据结构操作。
 - 解决问题（算法）需要选择正确的数据结构

§ 1.1 算法的基本概念（小结）

- 算法
 - 有限条指令的序列
 - 这个指令序列解决了某个问题的一系列运算或操作
- 算法A解决问题P
 - 把问题P的任何实例作为算法A的输入
 - 每步计算是确定性的
 - A能够在有限步停机
 - 输出该实例的正确的解

- 本章主要知识点:

§ 1.1 算法的基本概念

§ 1.2 算法描述 (回顾)

§ 1.3 算法分析

§ 1.2 算法描述（算法的伪码表示）

- 用伪码表示算法
 - 伪码不是程序代码，只是给出算法的主要步骤
 - 使用最清晰、最简洁的表示方法（包括自然语言）来说明给定的算法
 - 伪码中允许过程调用
- 算法的伪码描述：
 - 赋值语句：←
 - 分支语句：if ... then ... [else ...]
 - 循环语句：while, for, repeat until
 - 输出语句：return
 - 调用：直接写过程的名字
 - 注释：//...

§ 1.2 算法描述（算法的伪码表示）

- 算法的伪码描述（举例）
 - 二分归并排序伪代码

Algorithm 1. MergeSort(A, p, r)	
输入：数组 $A[p..r]$	
输出：按递增顺序排序的数组 A	
1	If $p < r$
2	Then $q \leftarrow \lfloor (p+r)/2 \rfloor$
3	MergeSort(A, p, q)
4	MergeSort($A, p+1, r$)
5	Merge(A, p, q, r)

§ 1.2 算法描述（续）

- 描述方式的约定（举例）
 - 采用C++语言描述
 - 类型丰富、语句精炼
 - 具有面向过程和面向对象的双重特点
 - 采用C++语言描述算法可使整个算法结构紧凑，可读性强
 - 采用C++语言与自然语言结合的方式描述算法

§ 1.2 算法描述（续）

- 抽象数据类型，是算法的一个数据模型连同定义在该模型上并作为算法构件的一组运算。
 - 举例：A.length; A[i].key
- 它带给算法设计的好处有：
 - 算法顶层设计与底层实现分离；
 - 算法设计与数据结构设计隔开，允许数据结构自由选择；
 - 便于空间和时间耗费的折衷；
 - 用抽象数据类型表述的算法具有很好的可维护性；
 - 算法自然呈现模块化；
 - 为自顶向下逐步求精和模块化提供有效途径和工具；
 - 算法结构清晰，层次分明，便于算法正确性的证明和复杂性的分析。

- 本章主要知识点：
 - § 1.1 算法的基本概念
 - § 1.2 算法描述
 - § 1.3 算法分析

§ 1.3 算法分析

— 计算机程序性能和资源使用的理论分析

● 为什么要进行算法分析

— 例： 对于排序问题（问题规模： $n=10^6$ ）

插入排序算法：复杂度 $c_1 n^2$

合并排序算法：复杂度 $c_2 n \log_2 n$

计算机A每秒能执行10亿条指令

计算机B每秒能执行1000万条指令

计算机A+插入排序算法（ $c_1=2$ ），则计算机A花的时间为：

$$\frac{2 \cdot (10^6)^2 \text{ 条指令}}{10^9 \text{ 条指令 / 秒}} = 2000 \text{ 秒}$$

计算机B+合并排序算法（ $c_2=50$ ），则计算机B花的时间为：

$$\frac{50 \cdot 10^6 \lg 10^6 \text{ 条指令}}{10^7 \text{ 条指令 / 秒}} \approx 100 \text{ 秒}$$

排序问题（算法分析举例）

输入：数列 $\langle a_1, a_2, \dots, a_n \rangle$

输出：排列 $\langle a'_1, a'_2, \dots, a'_n \rangle$ 使得

$$a'_1 \leq a'_2 \leq \dots \leq a'_n$$

例如：

输入： 8 2 4 9 3 6

输出： 2 3 4 6 8 9

插入排序

“伪代码”

INSERTION-SORT (A, n) $\triangleright A[1 \dots n]$

for $j \leftarrow 2$ to n

do $key \leftarrow A[j]$

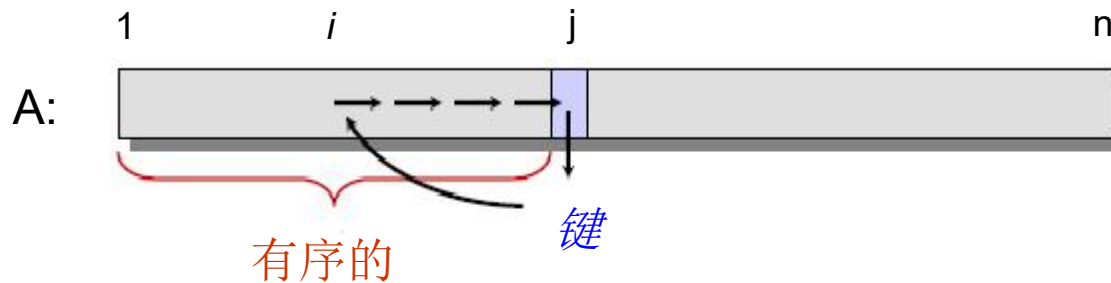
$i \leftarrow j - 1$

while $i > 0$ and $A[i] > key$

do $A[i+1] \leftarrow A[i]$

$i \leftarrow i - 1$

$A[i+1] = key$



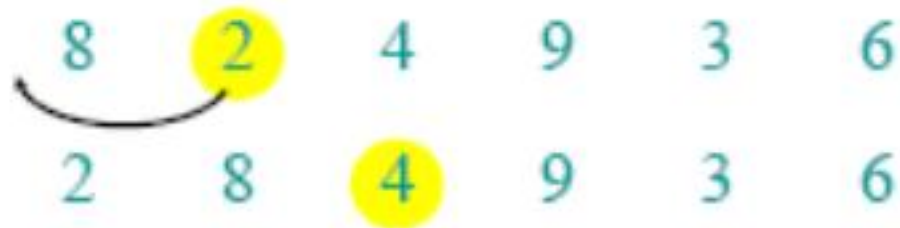
插入排序例子

8 2 4 9 3 6

插入排序例子



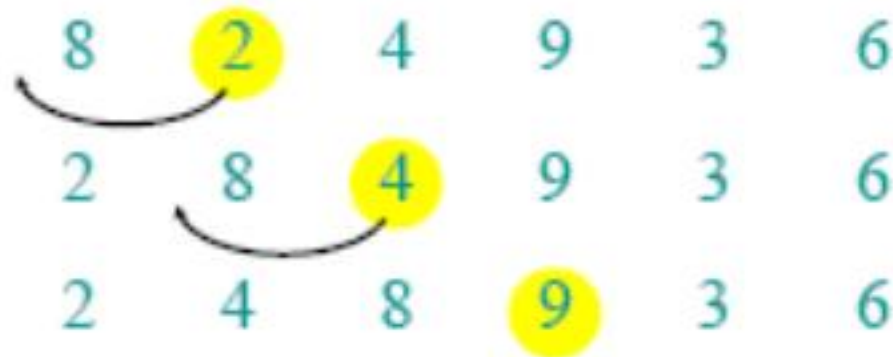
插入排序例子



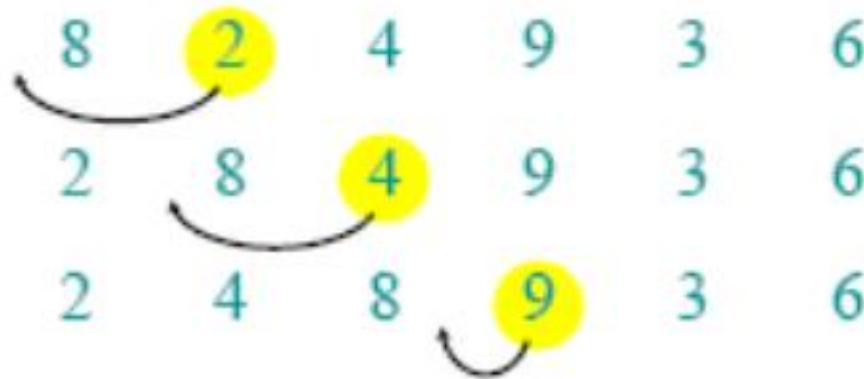
插入排序例子



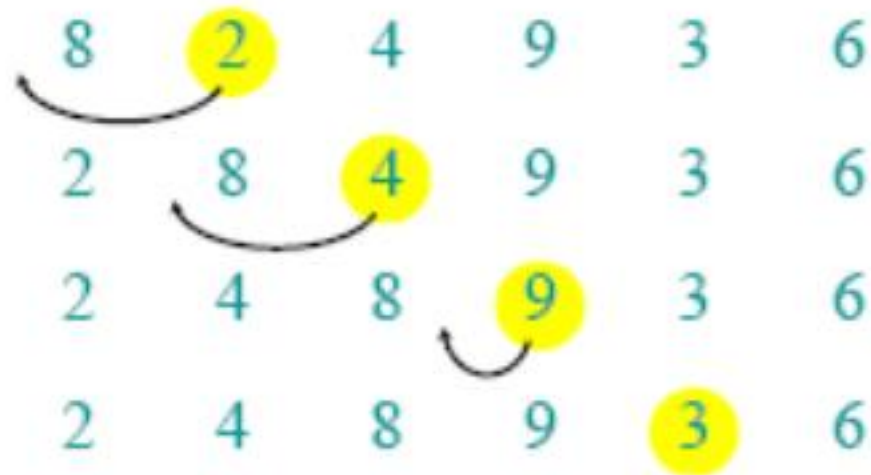
插入排序例子



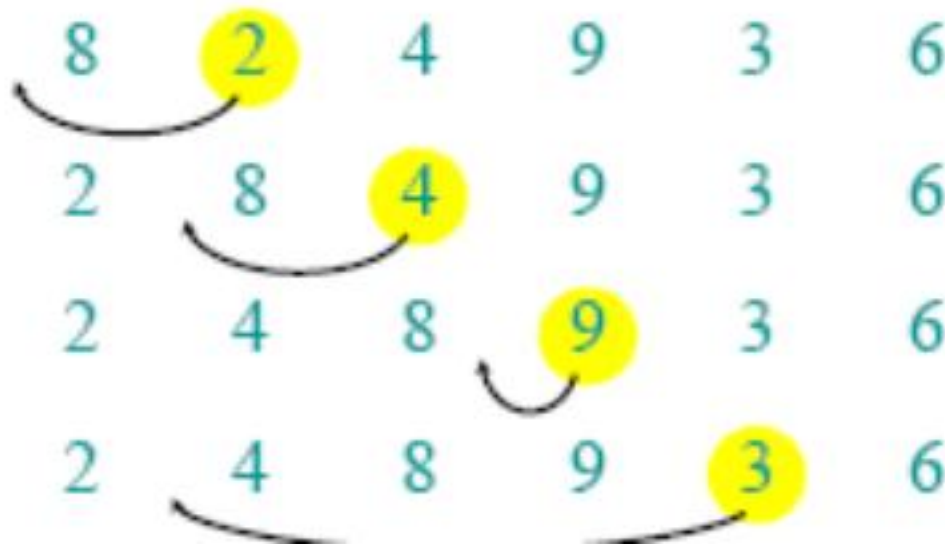
插入排序例子



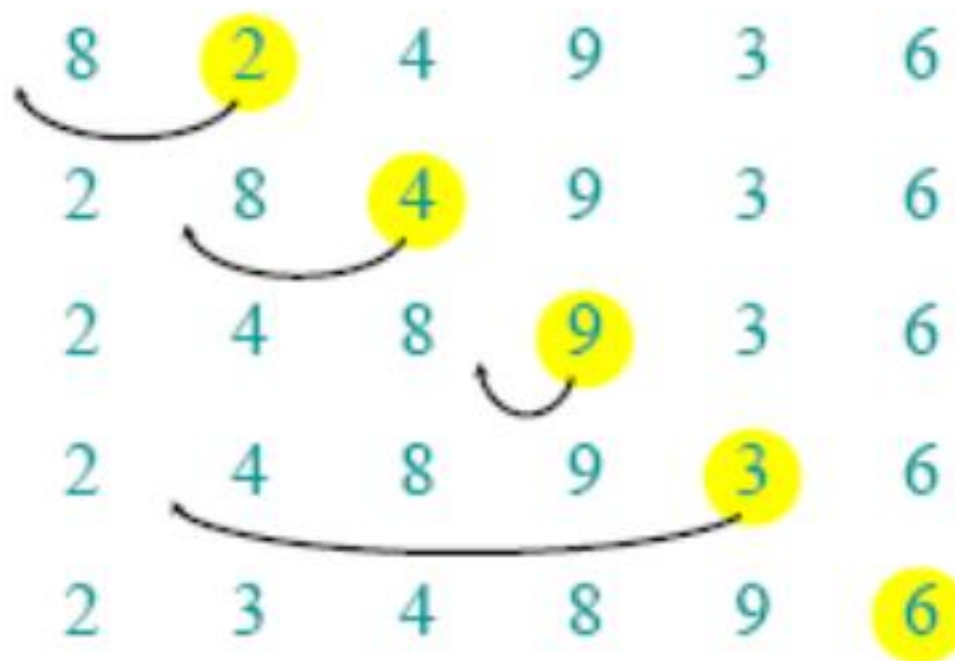
插入排序例子



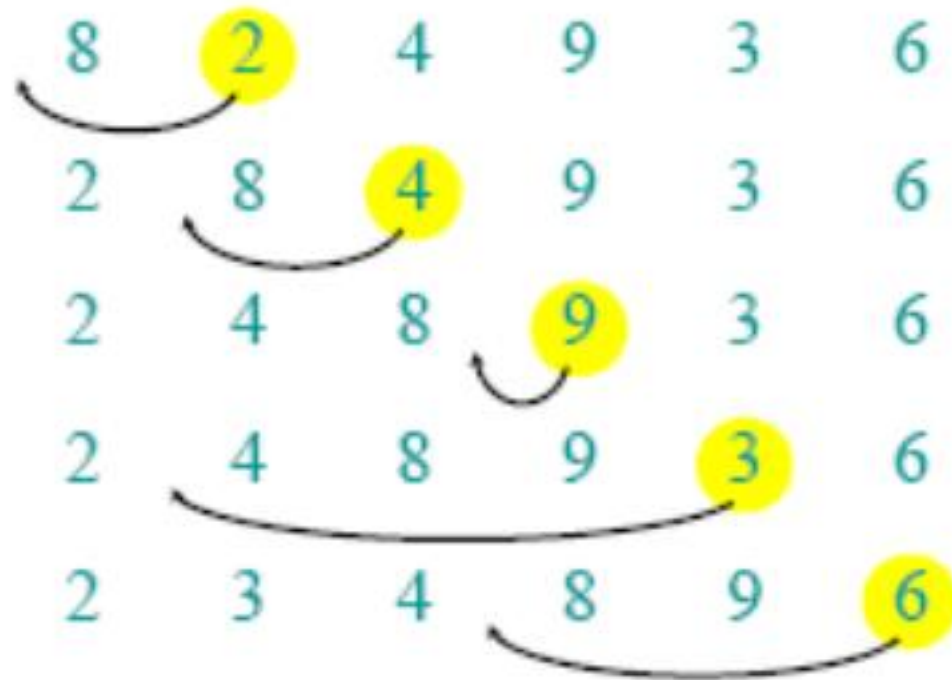
插入排序例子



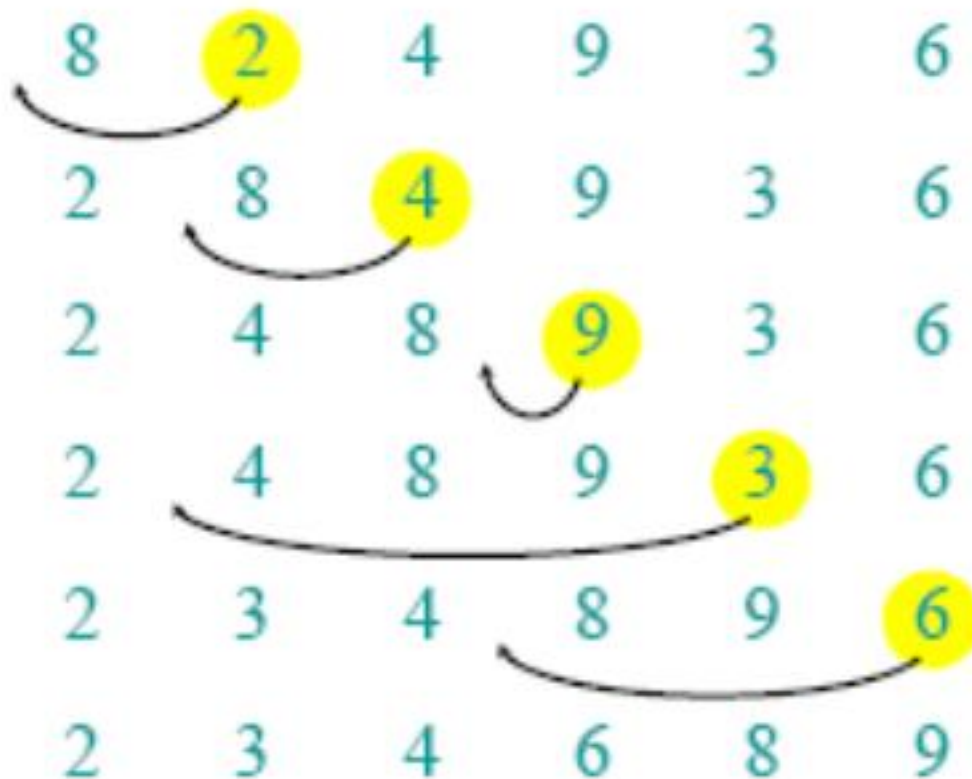
插入排序例子



插入排序例子



插入排序例子



完成

- 运行时间依赖于输入：已经排序的序列更容易排序。
- 运行时间依赖于输入的序列的规模，小的序列比大序列更容易排序。
- 总之，我们寻找运行时间的上界，因为每个人都喜欢有保证。

最坏—情况：（通常）

- $T(n)$ = 任何输入大小为 n 的情况下算法的最大运行时间

平均—情况：（有时）

- $T(n)$ = 所有输入大小为 n 的情况下算法的期望运行时间
- 需要假设输入的统计分布

最好—情况：（骗人的）

- 即使一个很慢的算法在 **某些** 输入下也会运行很快

插入排序最坏情况的运行时间是多少？

- 它依赖于计算机的速度：
 - 相对速度（在同一计算机上）
 - 绝对速度（在不同计算机上）
- 好主意：
 - 忽略计算机相关的常数
 - 看 $n \rightarrow \infty$ 时 $T(n)$ 的增长

“渐进分析”

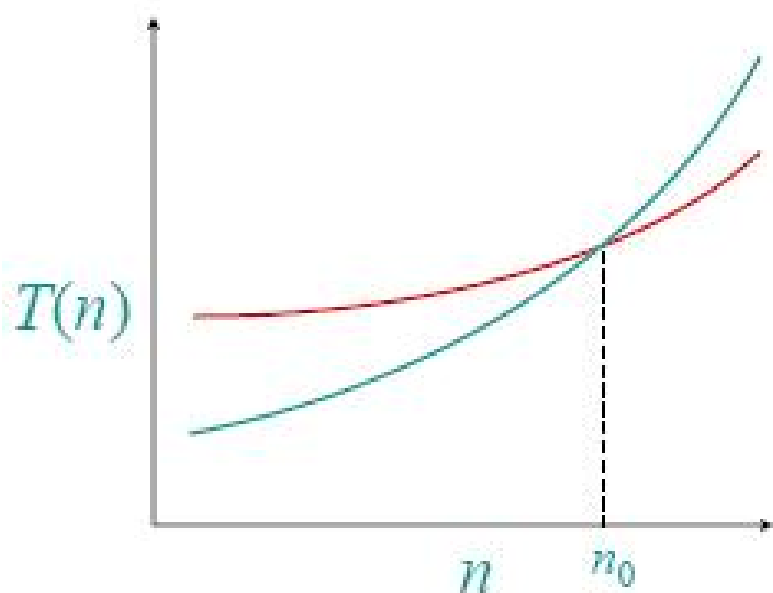
数学：

$\Theta(g(n)) = \{f(n) : \text{存在正常数 } c_1, c_2 \text{ 和 } n_0, \text{ 使得}$
对所有的 $n \geq n_0$, 有 $0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)\}$

工程：

- 丢弃低阶项，忽略首项的常数
- 例子： $3n^3 + 90n^2 - 5n + 6046 = \Theta(n^3)$

- 当 n 充分大的时候, $\Theta(n^2)$ 算法总是比 $\Theta(n^3)$ 算法好



- 我们不应该忽略渐进比较慢的算法
- 真实的设计环境通常要在工程目标上作仔细的权衡
- 渐进分析是帮助我们思考的有效工具

最坏情况：输入逆序

$$T(n) = \sum_{j=2}^n \Theta(j) = \Theta(n^2) \quad [\text{数学数列}]$$

平均情况：各种排列的概率相同

$$T(n) = \sum_{j=2}^n \Theta(j/2) = \Theta(n^2)$$

插入排序是一种快的排序方法吗？

- 对比较小的 n ，差不多是。
- 对比较大的 n ，不是。

§ 1.3 算法分析

- 目的：
 - 分析算法就是估计算法所需资源（时间，空间，通信带宽等），以便选取有效的算法。
- 计算模型：
 - 单处理机，随机存取机（Random-Access Machine, RAM）计算模型，其中指令是顺序执行的，无并发操作
- 涉及的知识基础：
 - 离散组合数学、概率论、代数等（分析）、程序设计、数据结构（算法设计）

- 基本概念
 - 时间复杂性，是算法运行所需要的时间资源的量。
 - 空间复杂性，是算法运行所需要的空间资源的量。
 - 算法的复杂性依赖于算法要解的问题的规模、算法的输入和算法本身的函数。
 - 如果分别用 N 、 I 和 A 表示算法要解问题的规模、算法的输入和算法本身，而且用 C 表示复杂性，那么，应该有 $C=F(N,I,A)$ 。
 - 一般，把时间复杂性和空间复杂性分开，并分别用 T 和 S 来表示，则有： $T=T(N,I)$ 和 $S=S(N,I)$ 。（通常，让 A 隐含在复杂性函数名当中）

- 时间复杂性分析

- 算法耗费的时间与输入实例的大小、实例的构成有关

- 例如：插入排序就如同打牌时整理纸牌，其时间复杂性与纸牌的数量、牌的顺序有关

- 输入实例大小 **Input-Size**：通常用整数表示，取决于被研究的问题

- 例：排序一个数组，**Size**的自然度量是项数
两数相乘，最好的度量是总的位数

- 有时，需用两个或多个整数表示输入规模，如图（顶，边）

§ 1.3 算法分析（续）

— 运行时间：一台抽象的计算机上运行所需要的时间

- 用基本操作的数目（执行步数）来度量；（好处是算法分析独立于机器，即任何基本操作看作是单位时间）
- 用更接近实际的计算机上实现的时间来度量；（如RAM模型，不同的指令具有不同的执行时间）
但两者相差一个常数因子。

— 复杂性函数的具体化

- 设抽象计算机所提供的基本运算或操作有 k 个，它们分别记为 $O_1, \dots, O_k$ ，相应的运行时间和运行次数为 $t_1, \dots, t_k$ 和 $e_1, \dots, e_k$ ，则

$$T(N, I) = \sum_{i=1}^k t_i e_i(N, I)$$

§ 1.3 算法分析（续）

— 最坏情况、最好情况时间复杂性与平均时间复杂性

- 最坏情况时间复杂性：

最坏运行时间指Size为N时任何输入的最长运行时间。

$$T_{\max}(N) = \max_{I \in D_N} T(N, I) = \max_{I \in D_N} \sum_{i=1}^k t_i e_i(N, I) = \sum_{i=1}^k t_i e_i(N, I^*) = T(N, I^*)$$

- 最好情况情况时间复杂性：

最好运行时间指Size为N时任何输入的最短运行时间。

$$T_{\min}(N) = \min_{I \in D_N} T(N, I) = \min_{I \in D_N} \sum_{i=1}^k t_i e_i(N, I) = \sum_{i=1}^k t_i e_i(N, \tilde{I}) = T(N, \tilde{I})$$

I^* 是 D_N 中使 T 达到 $T_{\max}(N)$ 的合法输入； $\tilde{I}$ 是 D_N 中使达到 $T_{\min}(N)$ 的合法输入

§ 1.3 算法分析（续）

- 平均时间复杂性：

平均运行时间指Size为N时任何输入的平均运行时间。

$$T_{\text{avg}}(N) = \sum_{I \in D_N} P(I) T(N, I) = \sum_{I \in D_N} P(I) \sum_{i=1}^k t_i e_i(N, I)$$

其中 D_N 是规模为N的合法输入的集合；而 $P(I)$ 是在算法的应用中出现输入I的概率。

— 关于最坏情况的时间分析：

为何要分析算法的最坏运行时间？

- ① 它是算法对于任何输入的运行时间的上界；
- ② 对于某些算法，最坏情况常常发生，如在DB中搜索一个并不存在的记录；
- ③ 平均时间往往和最坏时间相当（常数因子相同）

— 关于平均运行时间：

- 常常假定一个给定Size的所有输入是等概率的。实际上这种可能并不一定成立，但可以用随机化算法强迫它成立。有时平均时间和最坏时间不是同数量级，算法选择依据是，最好、最坏的概率较小时，尽量选择平均时间较小的算法。

§ 1.3 算法分析（续）

10 例1

INSERTION-SORT(A)

1 **for** $j \leftarrow 2$ **to** $\text{length}[A]$ **do**

2 $\text{key} \leftarrow A[j]$

3 //Insert $A[j]$ into the sorted $A[1..j-1]$

4 $i \leftarrow j-1$

5 **while** $i > 0$ and $A[i] > \text{key}$ **do**

6 $A[i+1] \leftarrow A[i]$

7 $i \leftarrow i-1$

8 $A[i+1] \leftarrow \text{key}$

Cost times

C1 n

C2 n-1

0 n-1

C4 n-1

C5 $\sum_{j=2}^n t_j$

C6 $\sum_{i=2}^n (t_j - 1)$

C7 $\sum_{j=2}^n (t_j - 1)$

C8 n-1

t_j 为第5
行中
while
循环所
做的测
试次数

§ 1.3 算法分析（续）

— 算法的总运行时间

$$T(n) = c_1 n + c_2 (n-1) + c_4 (n-1) + c_5 \sum_{j=2}^n t_j + c_6 \sum_{j=2}^n (t_j - 1) + c_7 \sum_{j=2}^n (t_j - 1) + c_8 (n-1)$$

最好情况：输入数组已经是排好序的

此时有， $t_j=1$ ，则

$$T(n) = c_1 n + c_2 (n-1) + c_4 (n-1) + c_5 (n-1) + c_8 (n-1) = an + b$$

最坏情况：输入数组是按照逆序排序的

此时有， $t_j=j$ ，则

$$T(n) = c_1 n + c_2 (n-1) + c_4 (n-1) + c_5 \sum_{j=2}^n j + c_6 \sum_{j=2}^n (j-1) + c_7 \sum_{j=2}^n (j-1) + c_8 (n-1) = an^2 + bn + c$$

§ 1.3 算法分析

- 通常仅针对**指定基本运算**，计算算法所做的运算次数
- 基本运算：比较，加法，乘法，交换，...
- **输入规模**：输入串编码长度
 - 通常用下述参数度量：数组元素数目，调度问题的任务个数，图的顶点数与边数等
- 算法基本运算次数可表示为输入规模的函数
- 给定问题和基本运算就决定了一个算法类

—基本运算

- 排序：元素之间的比较
- 检索：被检索数组元素 x 与数组元素之间的比较
- 整数乘法：每位数字相乘（位乘）计1次
 - m 位和 n 位整数相乘要做 mn 次位乘
- 矩阵相乘：每对元素乘 计1次
- 图的遍历：置指针

§ 1.3 算法分析

—输入规模：

- 排序：数组中元素个数 n
- 检索：被检索数组的元素个数 n
- 整数乘法：两个整数的位数 m, n
- 矩阵相乘：矩阵的行列数 i, j, k
- 图的遍历：图的顶点数 n , 边数 m

§ 1.3 算法分析

—举例：检索问题

- 输入：非降顺序排列的数组 L ，元素数 n ，数 x
- 输入： j
- 若 x 在 L 中， j 是 x 首次出现的下标；否则 $j=0$
- 基本运算： x 与 L 中元素的比较

§ 1.3 算法分析

—举例：检索问题

—顺序检索算法

- $j=1$, 将 x 与 $L[j]$ 比较. 如果 $x=L[j]$, 则算法停止, 输入 j ; 如果不等, 则把 j 加 1, 继续 x 与 $L[j]$ 的比较, 如果 $j>n$, 则停机并输出 0

1	2	3	4	5
---	---	---	---	---

- $x=4$, 需要比较 4 次
- $x=2.5$, 需要比较 5 次

§ 1.3 算法分析

—举例：检索问题

—最坏情况的时间估计

- 不同的输入有 $2n+1$ 个，分别对应
- $x=L[1], x=L[2], \dots, x=L[n]$
- $x < L[1], L[1] < x < L[2], L[2] < x < L[3], \dots, L[n] < x$
- 最坏情况下时间： $W(n) = n$ （要做 n 次比较）
- 最坏的输入： x 不在 L 中或者 $x=L[n]$

§ 1.3 算法分析

—举例：检索问题

—平均情况的时间估计

- 输入实例的概率分布：假设 x 在 L 中概率是 p ，且每个位置概率相等

$$\begin{aligned} \bullet A(n) &= \sum_{i=1}^n i \frac{p}{n} + (1-p)n \\ &= \frac{p(n+1)}{2} + (1-p)n \end{aligned}$$

当 $p=1/2$ 时， $A(n) \approx 3n/4$

§ 1.3 算法分析

—举例：检索问题

—改进顺序检索算法

$j=1$, 将 x 与 $L[j]$ 比较. 如果 $x=L[j]$, 则算法停止, 输入 j ; 如果 $x>L[j]$, 则把 j 加 1, 继续 x 与 $L[j]$ 的比较; 如果 $x<L[j]$, 则停机并输出 0; 如果 $j>n$, 则停机并输出 0

问题：改进检索算法的最坏时间复杂度和平均时间复杂度是多少？

MERGE-SORT $A[1 \dots n]$

1. If $n = 1$, done.
2. Recursively sort $A[1 \dots \lceil n/2 \rceil]$
and $A[\lceil n/2 \rceil + 1 \dots n]$.
3. “*Merge*” the 2 sorted lists.

关键函数： **MERGE**

合并两个已排序数组

20 12

13 11

7 9

2

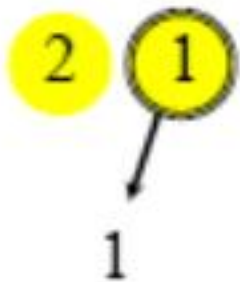
1

合并两个已排序数组

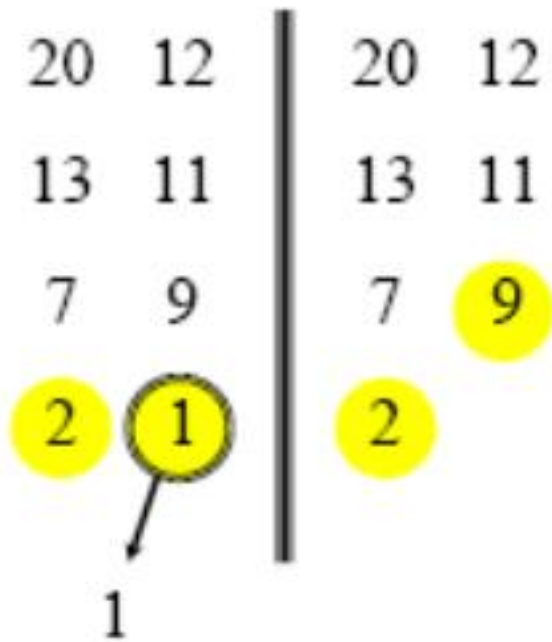
20 12

13 11

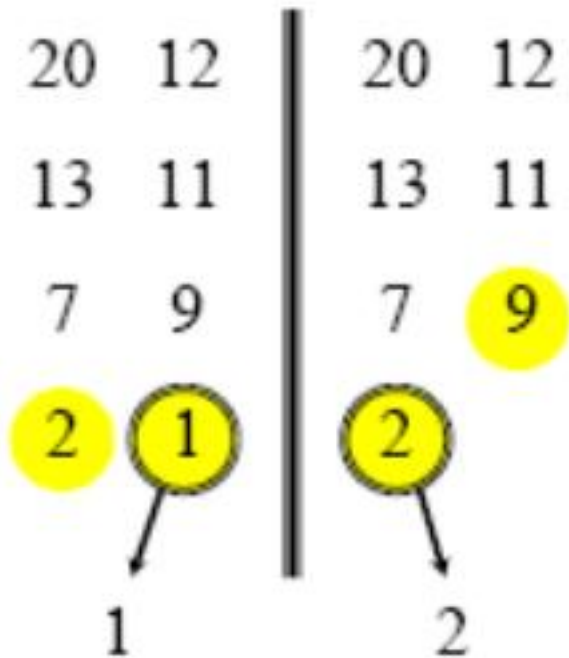
7 9



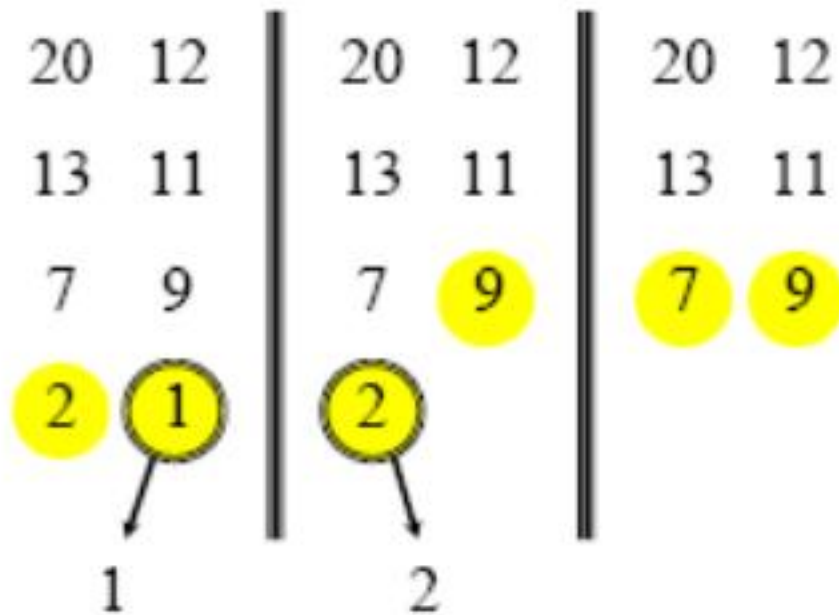
合并两个已排序数组



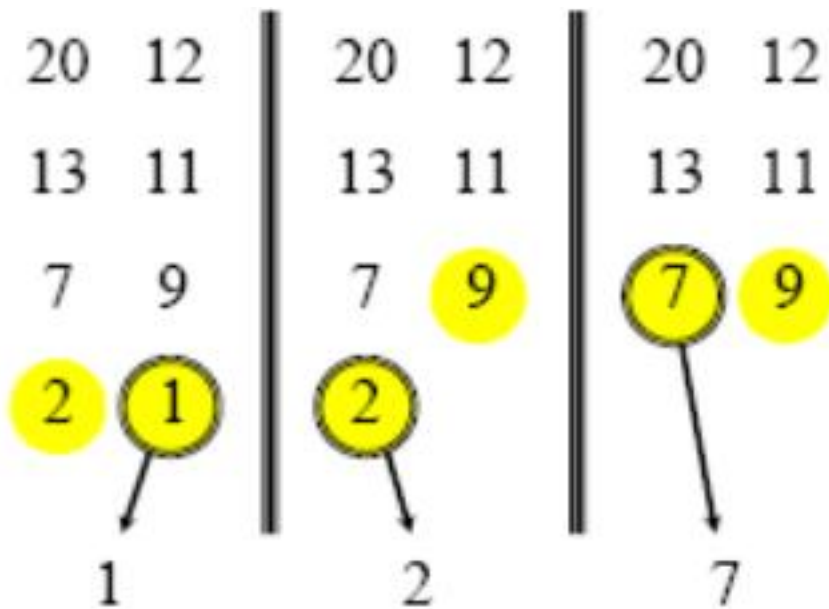
合并两个已排序数组



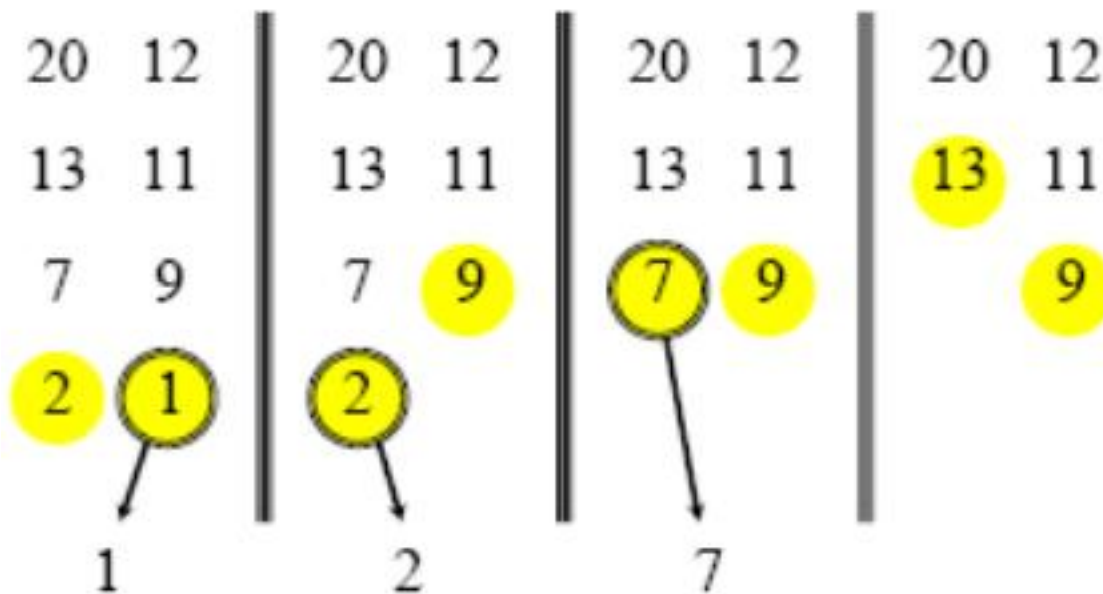
合并两个已排序数组



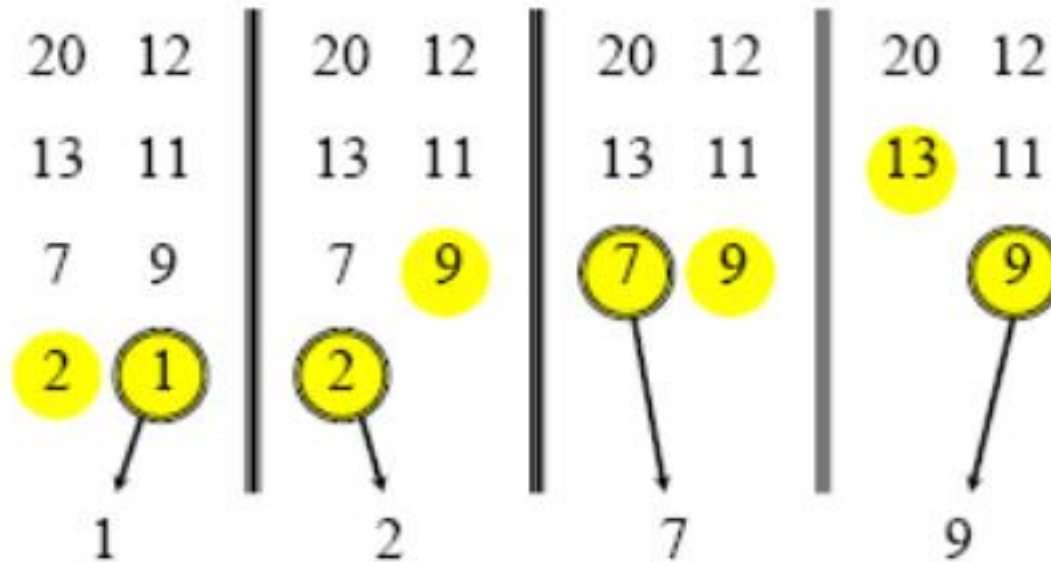
合并两个已排序数组



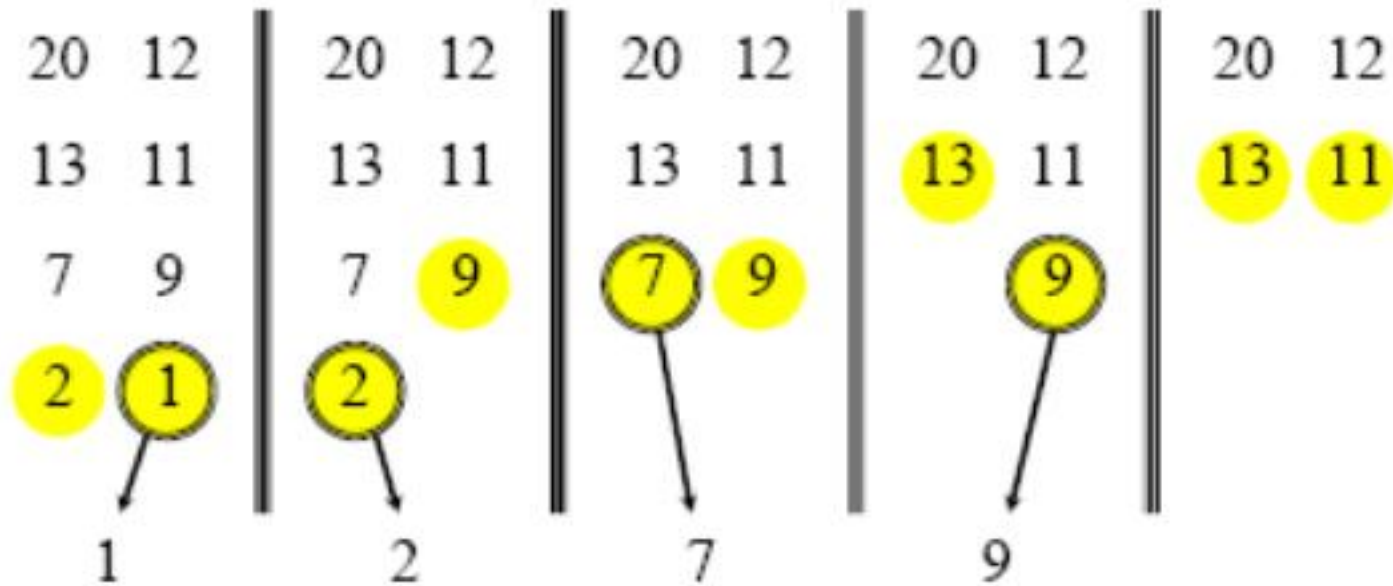
合并两个已排序数组



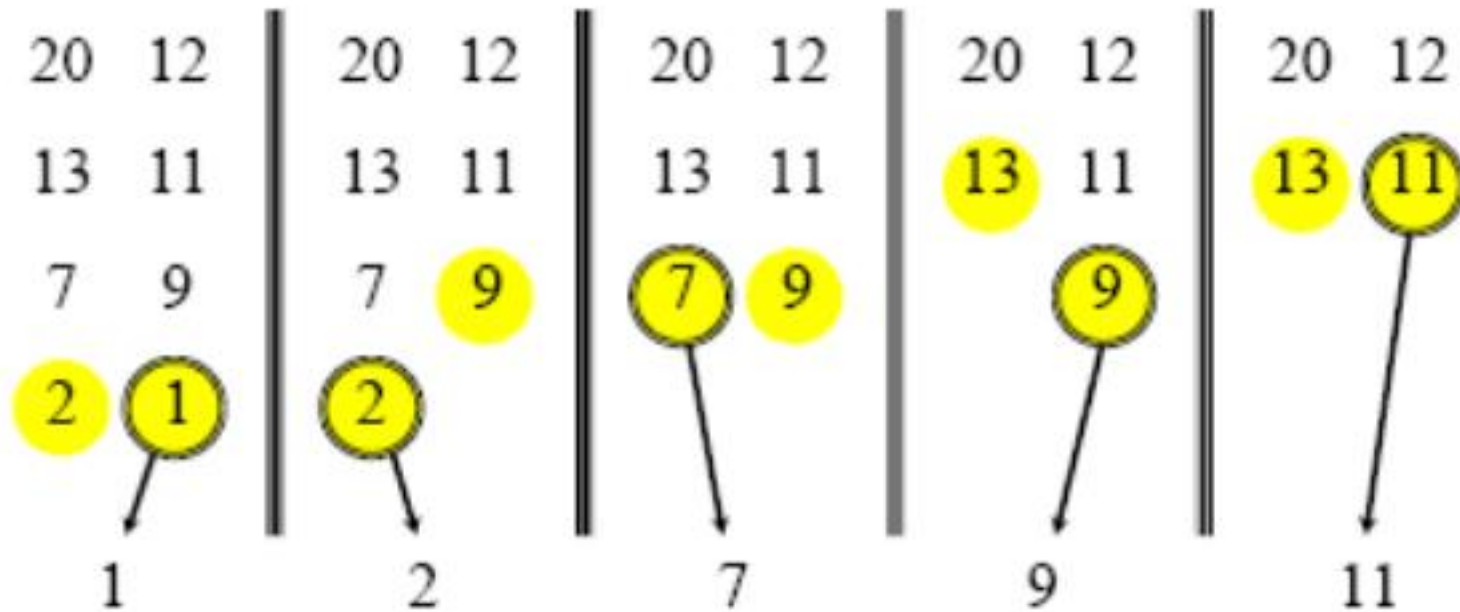
合并两个已排序数组



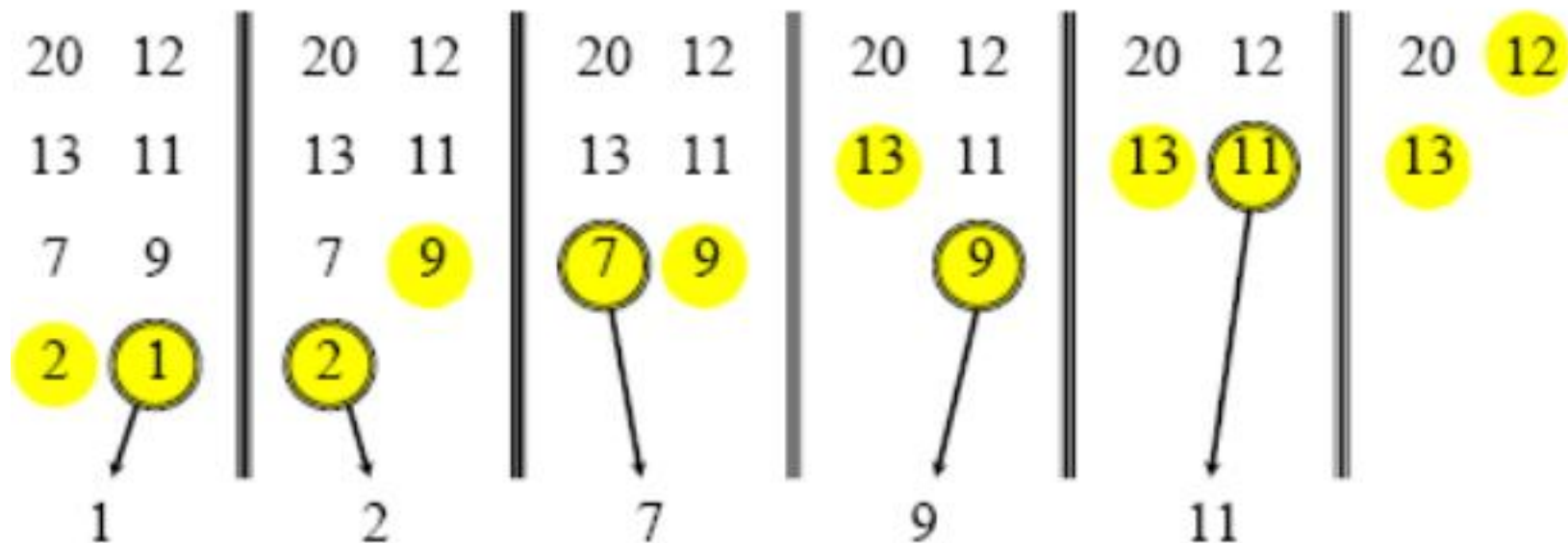
合并两个已排序数组



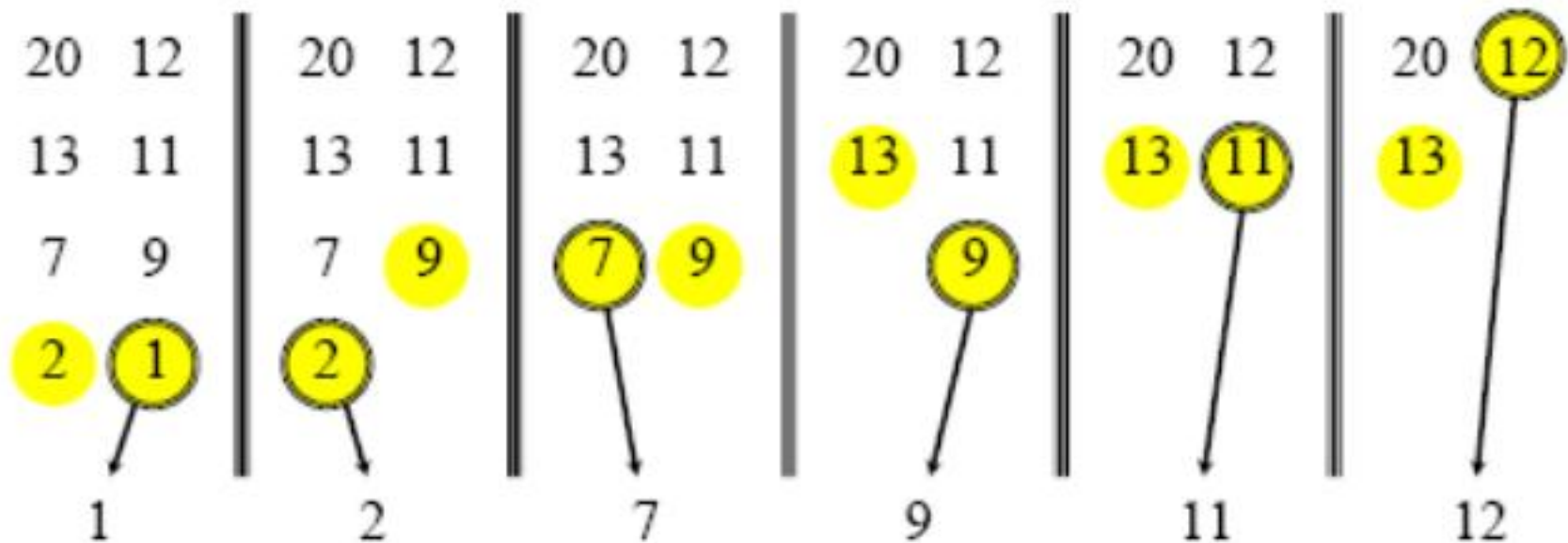
合并两个已排序数组



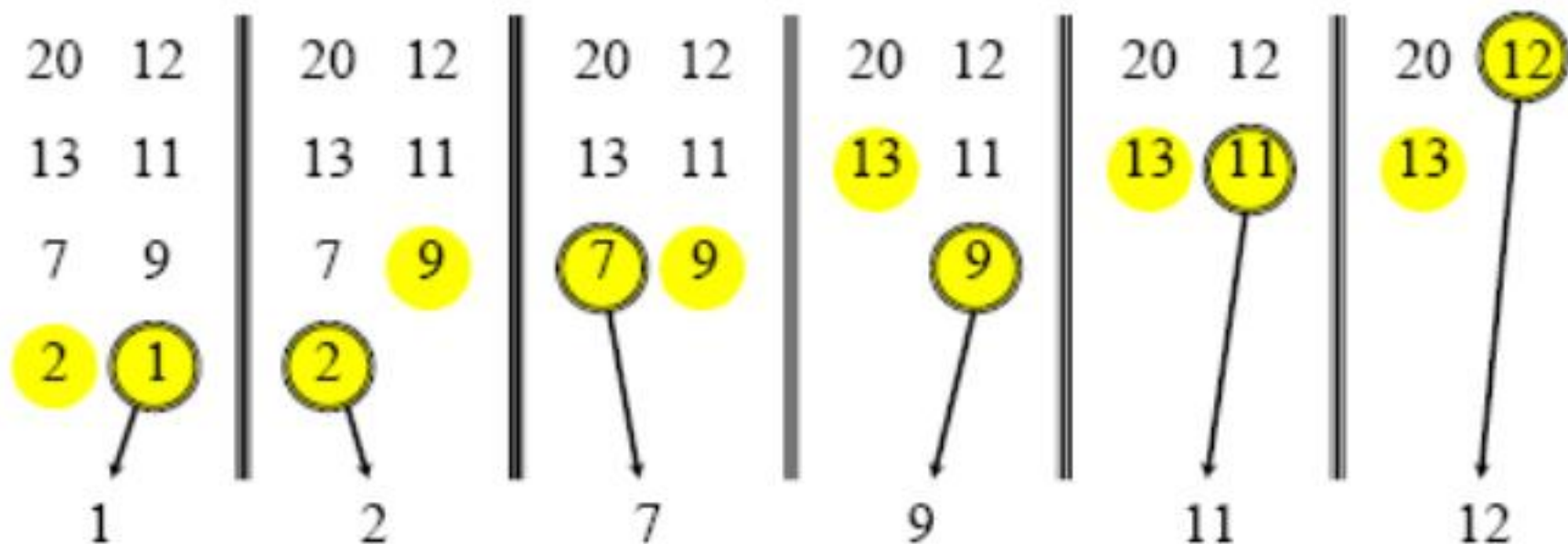
合并两个已排序数组



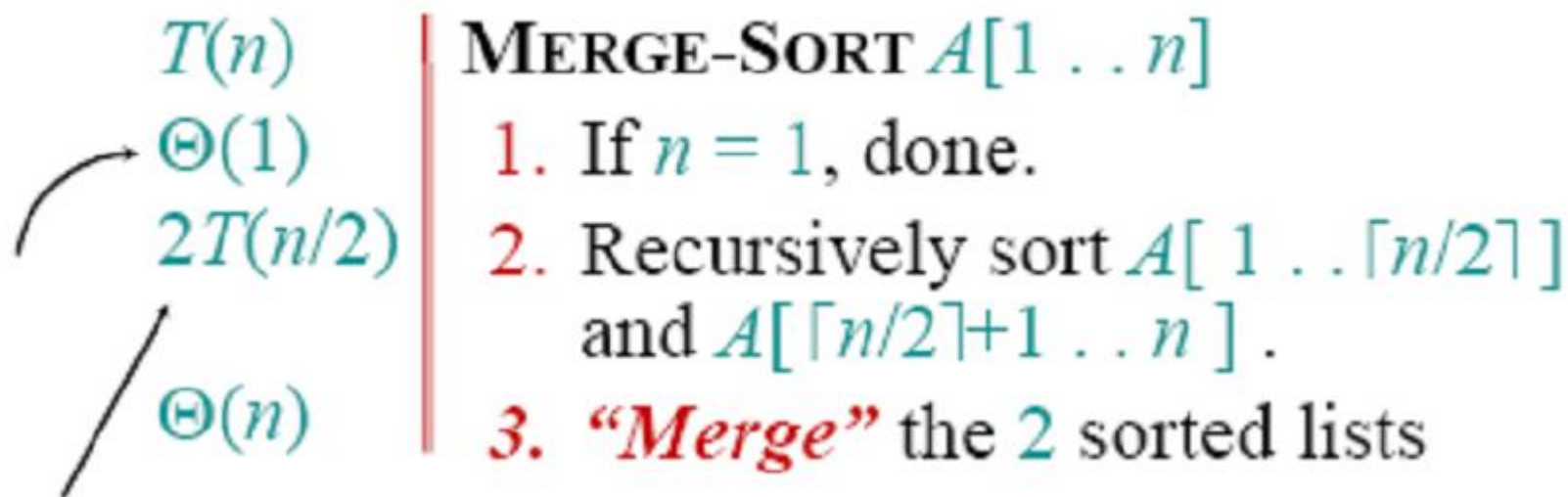
合并两个已排序数组



合并两个已排序数组



合并n个项需要时间 = $\Theta(n)$
(线性时间)。



此处不严谨：应该是 $T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor)$

但是对渐进分析没有影响

合并排序的递归表示

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1; \\ 2T(n/2) + \Theta(n) & \text{if } n > 1. \end{cases}$$

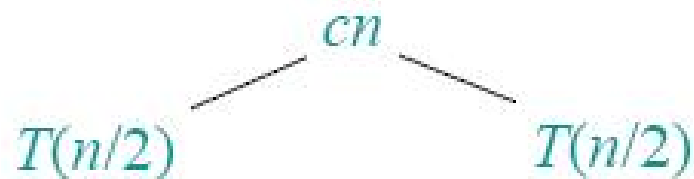
- 当 n 足够小的时候，我们通常忽略起始状态 $T(n) = \Theta(1)$ ，但仅仅是在它对递归的渐进分析的结果没有影响的时候。
- 找 $T(n)$ 上界的方法？

求解 $T(n) = 2T(n/2) + cn$, $c > 0$ 且为常数。

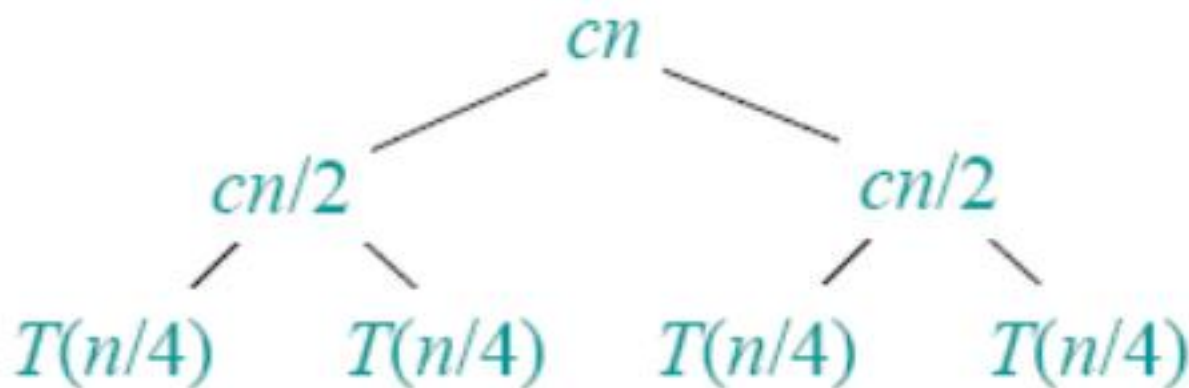
求解 $T(n) = 2T(n/2) + cn$, $c > 0$ 且为常数。

$$T(n)$$

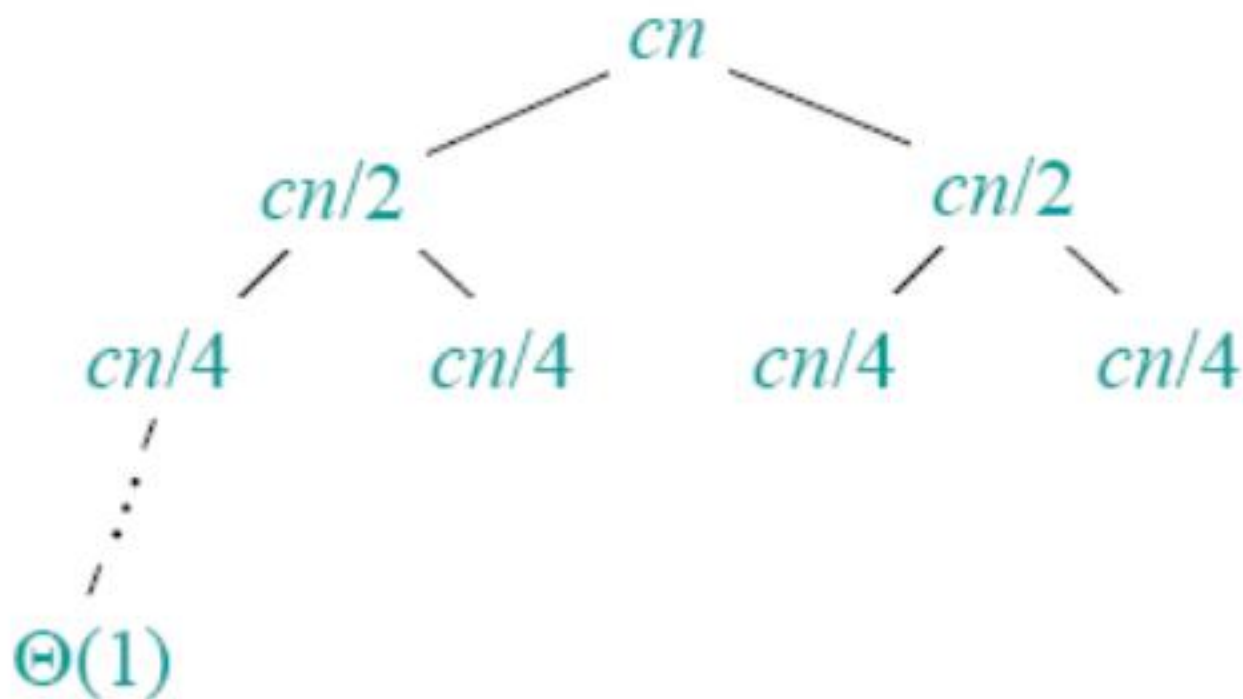
求解 $T(n) = 2T(n/2) + cn$, $c > 0$ 且为常数。



求解 $T(n) = 2T(n/2) + cn$, $c > 0$ 且为常数。

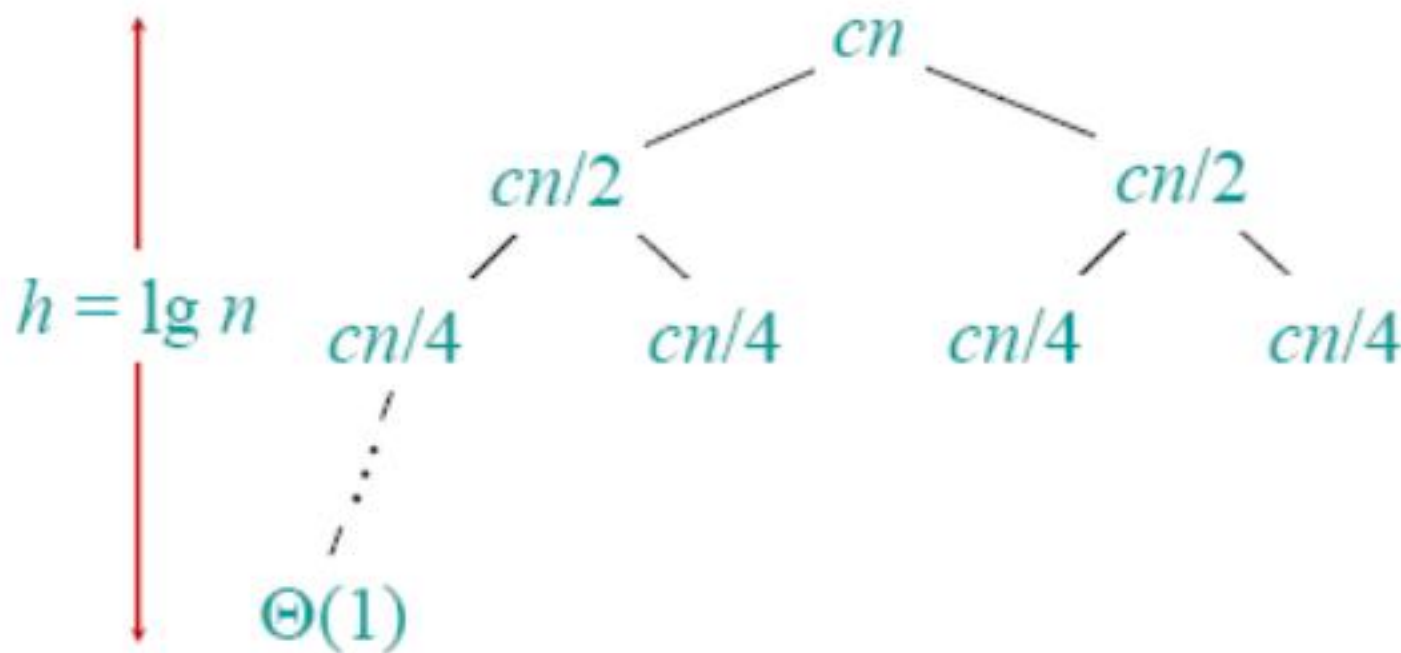


求解 $T(n) = 2T(n/2) + cn$, $c > 0$ 且为常数。



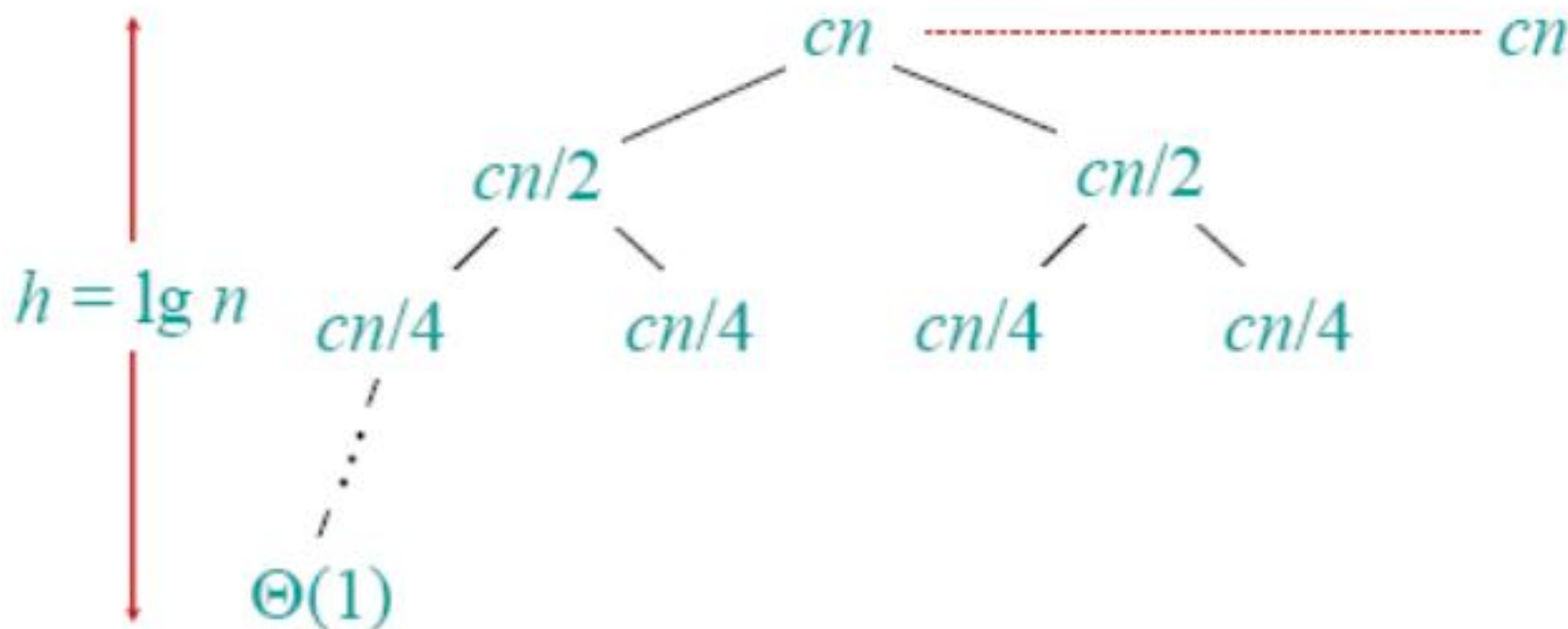
递归树

求解 $T(n) = 2T(n/2) + cn$, $c > 0$ 且为常数。



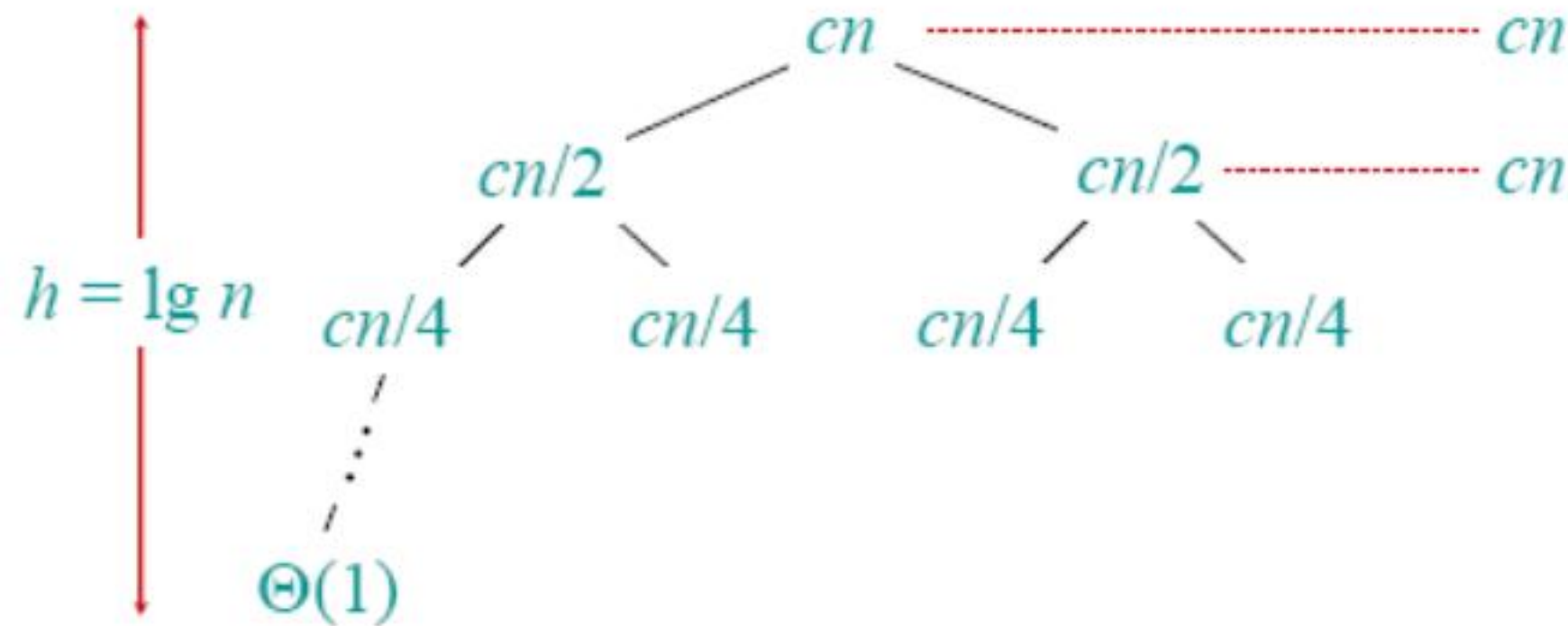
递归树

求解 $T(n) = 2T(n/2) + cn$, $c > 0$ 且为常数。



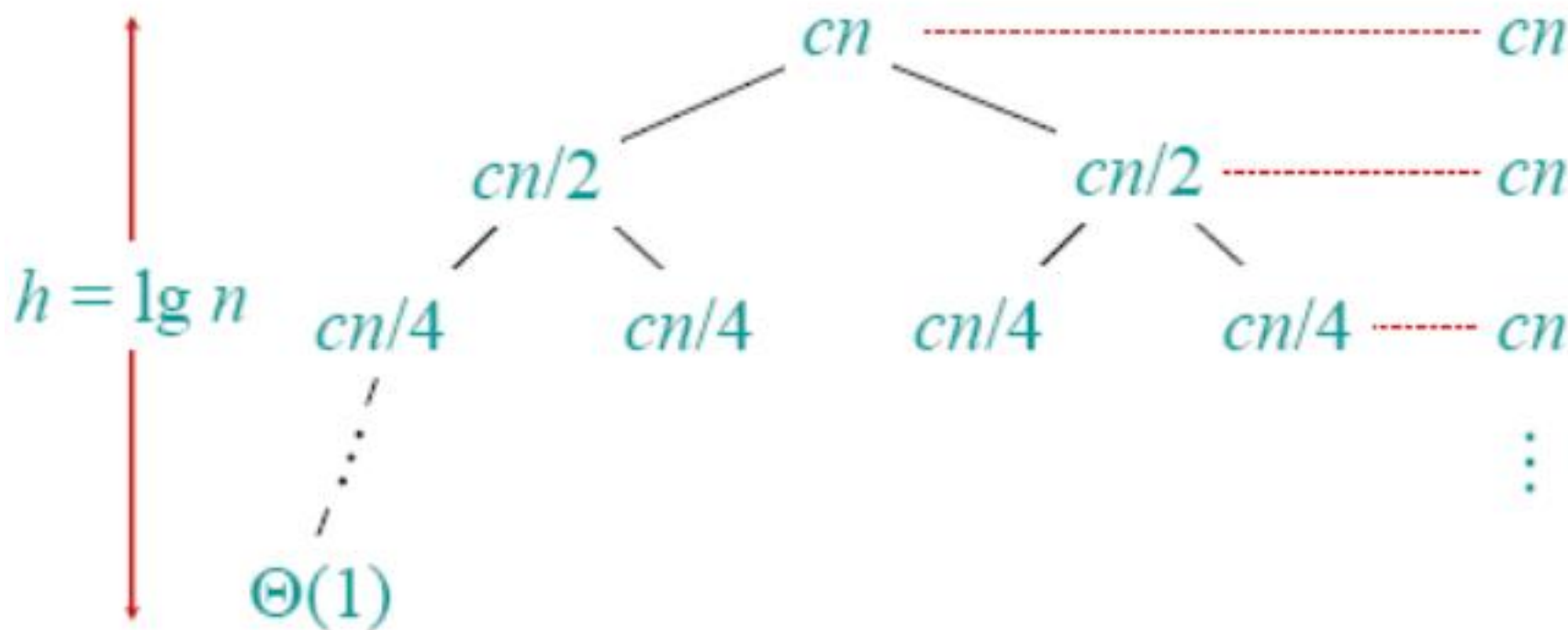
递归树

求解 $T(n) = 2T(n/2) + cn$, $c > 0$ 且为常数。



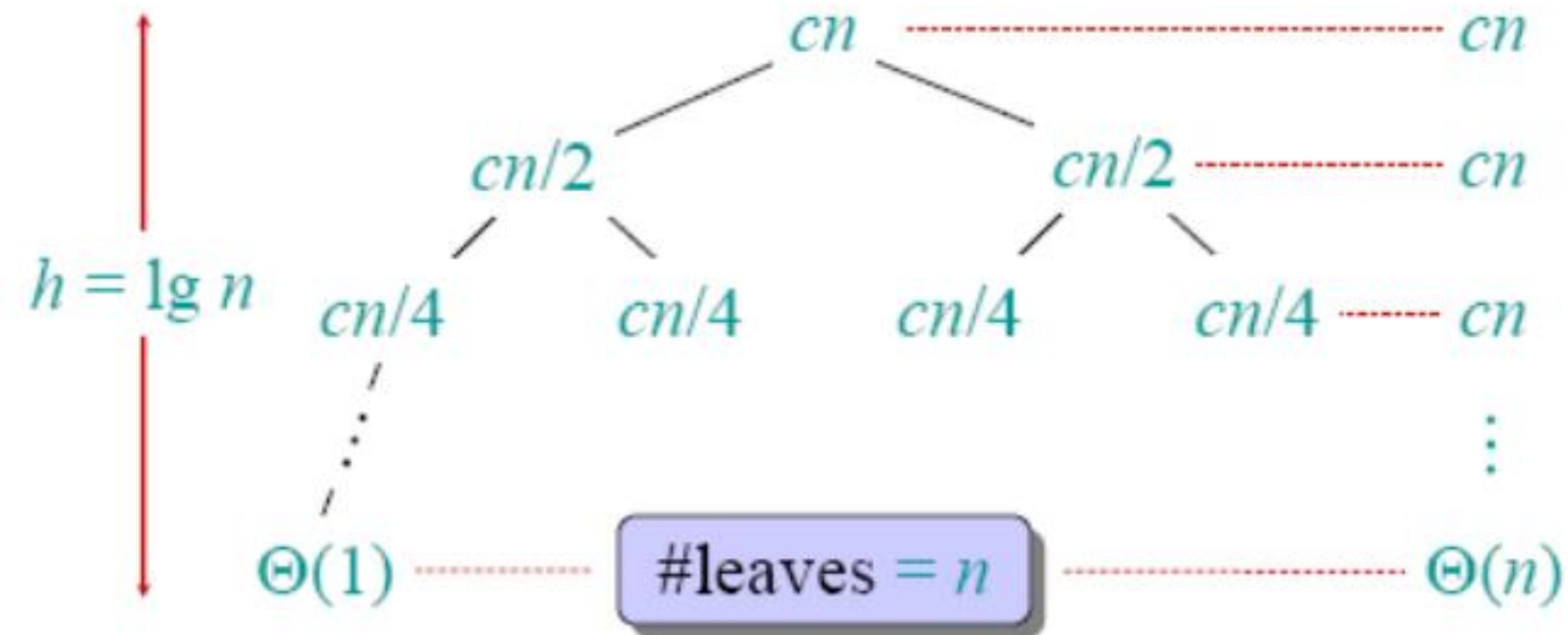
递归树

求解 $T(n) = 2T(n/2) + cn$, $c > 0$ 且为常数。



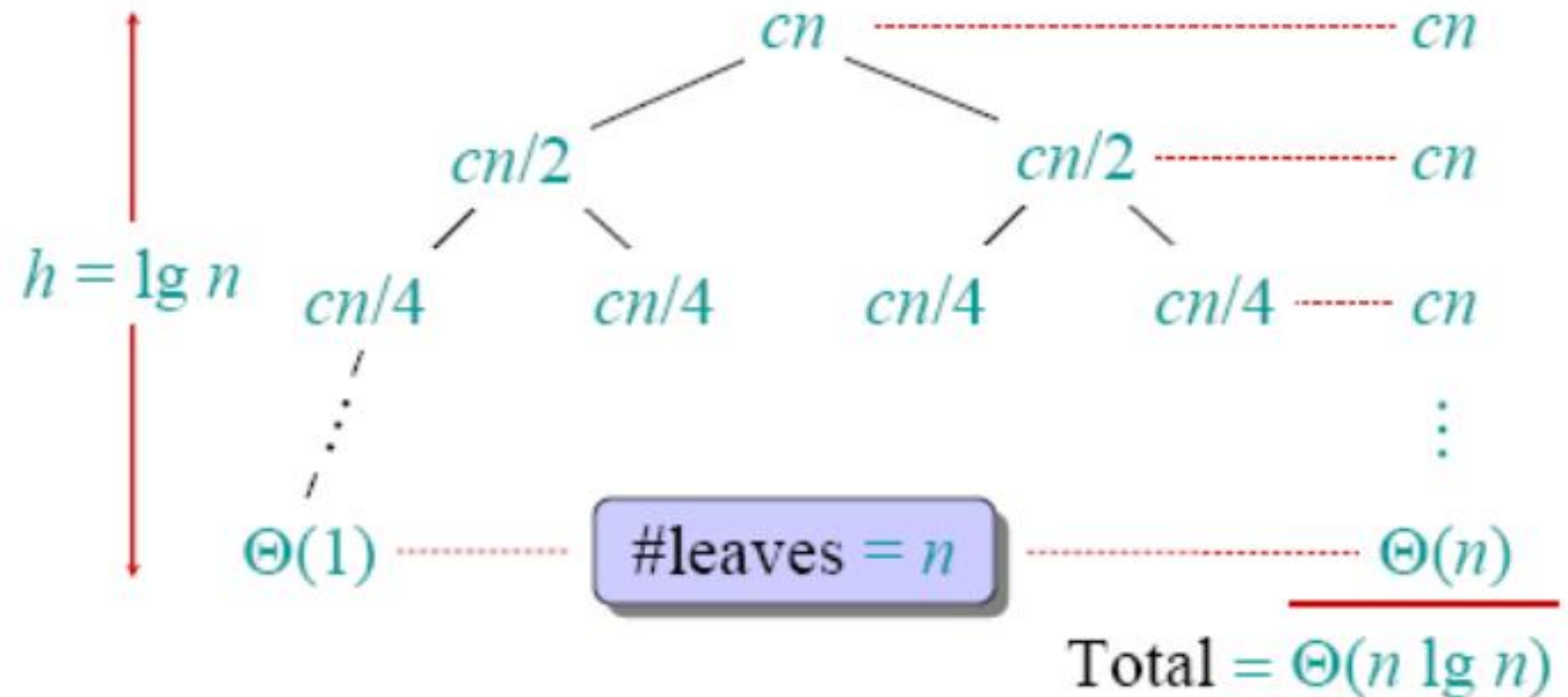
递归树

求解 $T(n) = 2T(n/2) + cn$, $c > 0$ 且为常数。



递归树

求解 $T(n) = 2T(n/2) + cn$, $c > 0$ 且为常数。



§ 1.3 算法分析

- $\Theta(n \lg n)$ 增长的比 $\Theta(n^2)$ 慢的多。
- 因此，合并排序的渐进性比插入排序要好
- 实际上，合并排序在 $n > 30$ 左右的时候开始性能比插入排序好
- 自己测试一下！

- 替代法
- 递归树
- 主方法

- 合并排序的分析需要求解一个递归式
- 求解递归式就像求解积分，微分方程一样。
学会一些技巧
- 递归式的应用

最通用的方法：

1. 猜测解的形式。
2. 通过推导验证。
3. 解出常数。

例子： $T(n) = 4T(n/2) + n$

- [假设 $T(1) = \Theta(1)$.]
- 猜测 $O(n^3)$. (分别证明 O 和 Ω .)
- 假设 $T(k) \leq ck^3$ 对于 $k < n$.
- 通过推导证明 $T(n) \leq cn^3$.

替代法举例

$$\begin{aligned}T(n) &= 4T(n/2) + n \\&\leq 4c(n/2)^3 + n \\&= (c/2)n^3 + n \\&= cn^3 - ((c/2)n^3 - n) \leftarrow \text{期望} - \text{余项} \\&\leq cn^3 \leftarrow \text{期望}\end{aligned}$$

$(c/2)n^3 - n \geq 0$, 例如, 如果 $c \geq 2$ 且 $n \geq 1$.

余项



- 我们必须处理初始条件，也就是说，首先要保证推导在初始情况成立。
- **初始：** 当 n_0 为适当的常量时，对于所有 $n < n_0$ $T(n) = \Theta(1)$ 都成立。
- 如果我们选择足够大的 c ，那么对于 $1 \leq n < n_0$ ，“ $\Theta(1)$ ” $\leq cn^3$.

边界并不紧密！

更接近的上界?

我们要证明 $T(n) = O(n^2)$.

假设 对于 $k < n$: $T(k) \leq ck^2$

$$T(n) = 4T(n/2) + n$$

$$\leq 4c(n/2)^2 + n$$

$$= cn^2 + n$$

$$= O(n^2) \quad \text{错误!}$$

$$= cn^2 - (-n) \quad [\text{期望} - \text{余项}]$$

$$\leq cn^2 \quad \text{没有任何 } c > 0 \text{ 满足. 失败!}$$

更接近的上界？

思想: 加强推导的假设.

- 减一个低阶项

推导假设: 对于 $k < n$, $T(k) \leq c_1 k^2 - c_2 k$.

$$\begin{aligned} T(n) &= 4T(n/2) + n \\ &= 4(c_1(n/2)^2 - c_2(n/2)) + n \\ &= c_1 n^2 - 2c_2 n + n \\ &= c_1 n^2 - c_2 n - (c_2 n - n) \\ &\leq c_1 n^2 - c_2 n \text{ 如果 } c_2 \geq 1. \end{aligned}$$

选择足够大的 c_1 使初始条件成立。

- 递归树对算法递归执行的花费（时间）建模
- 递归树方法可以用作替代法之前的猜想。
- 递归树方法可能不是很可靠
- 然而，递归树方法有启发的作用

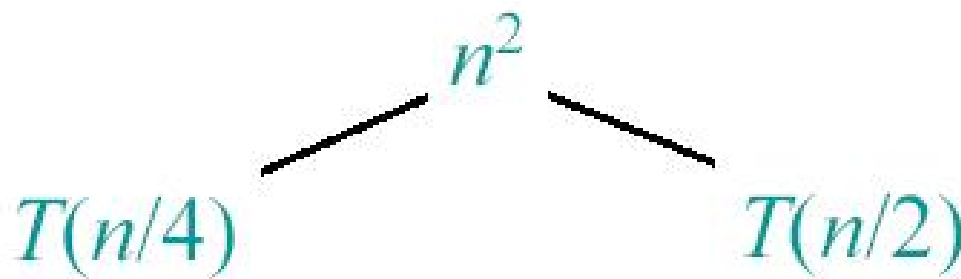
求解 $T(n) = T(n/4) + T(n/2) + n^2$:

求解 $T(n) = T(n/4) + T(n/2) + n^2$

$T(n)$

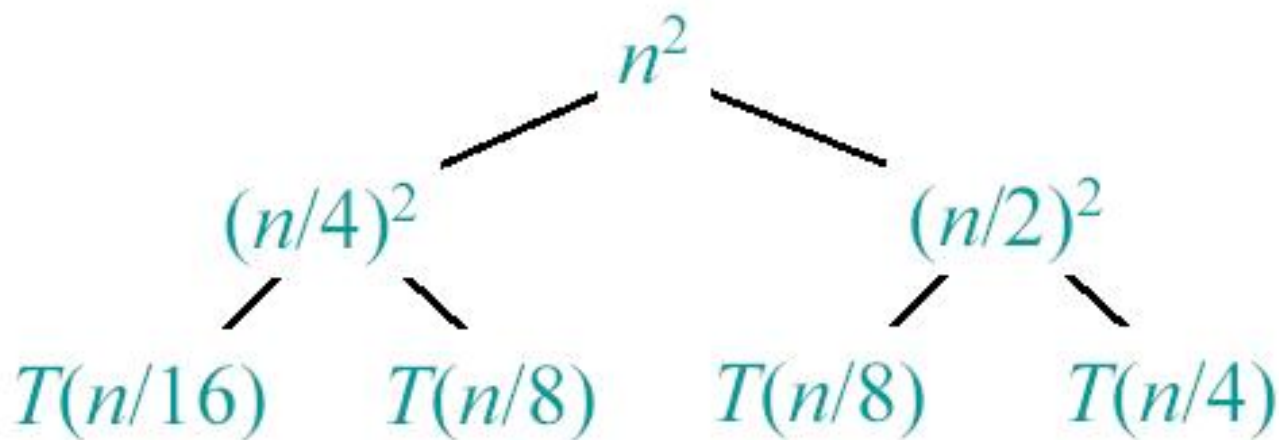
递归树方法举例

求解 $T(n) = T(n/4) + T(n/2) + n^2$:



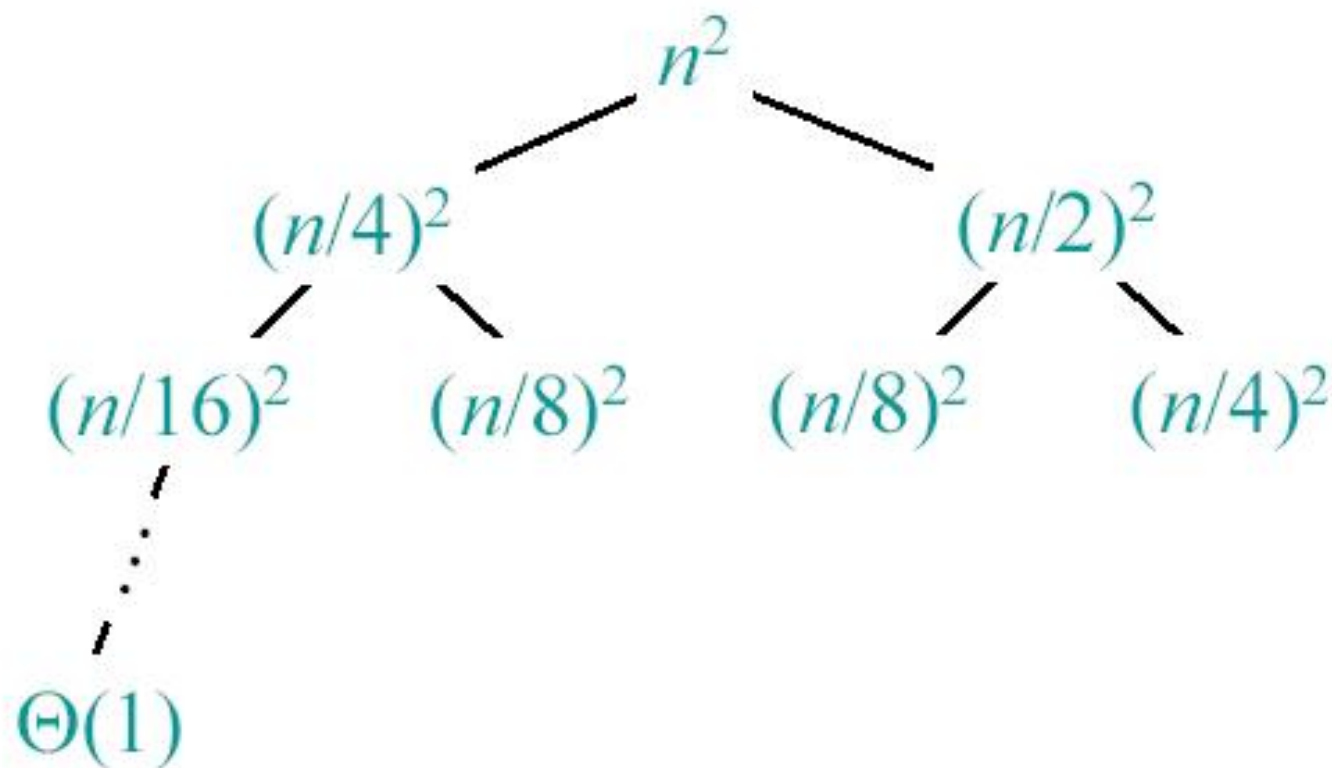
递归树方法举例

求解 $T(n) = T(n/4) + T(n/2) + n^2$:



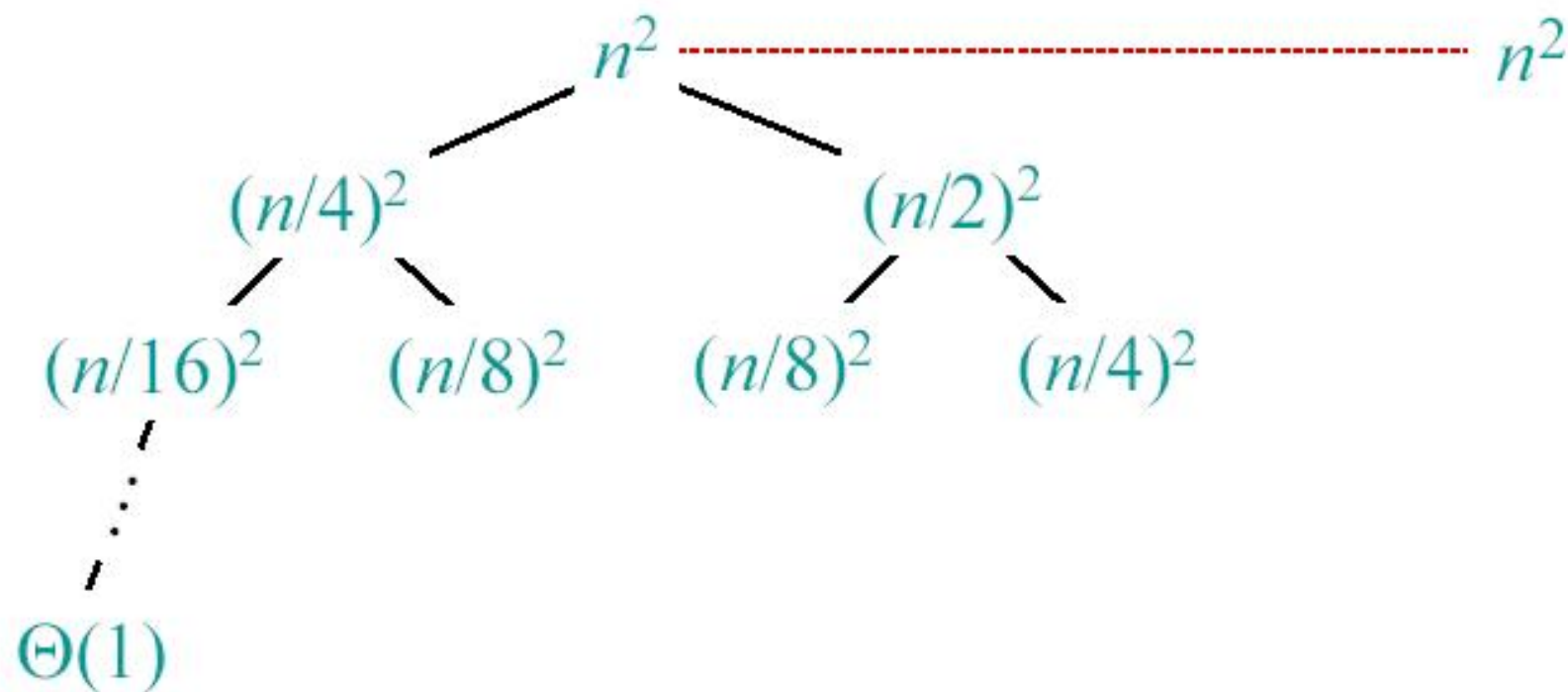
递归树方法举例

求解 $T(n) = T(n/4) + T(n/2) + n^2$:



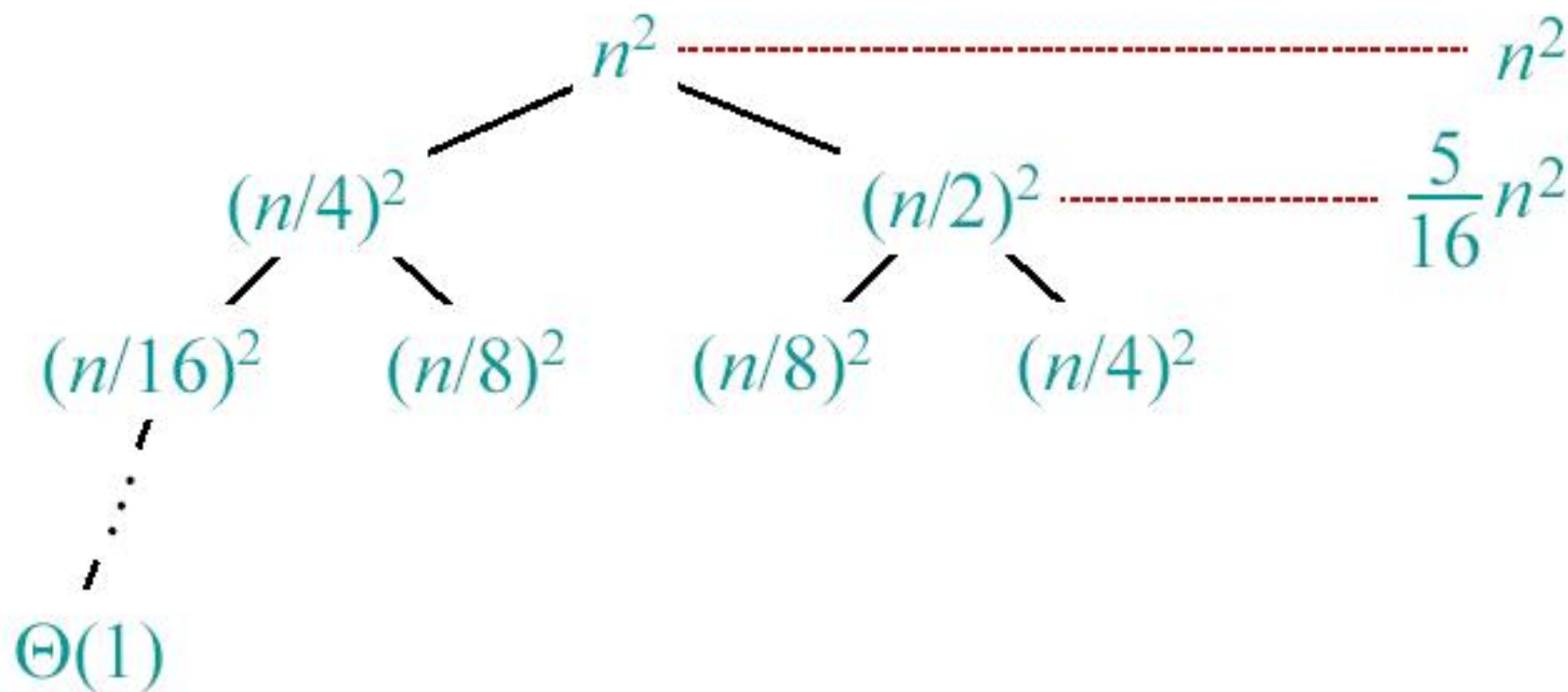
递归树方法举例

求解 $T(n) = T(n/4) + T(n/2) + n^2$:



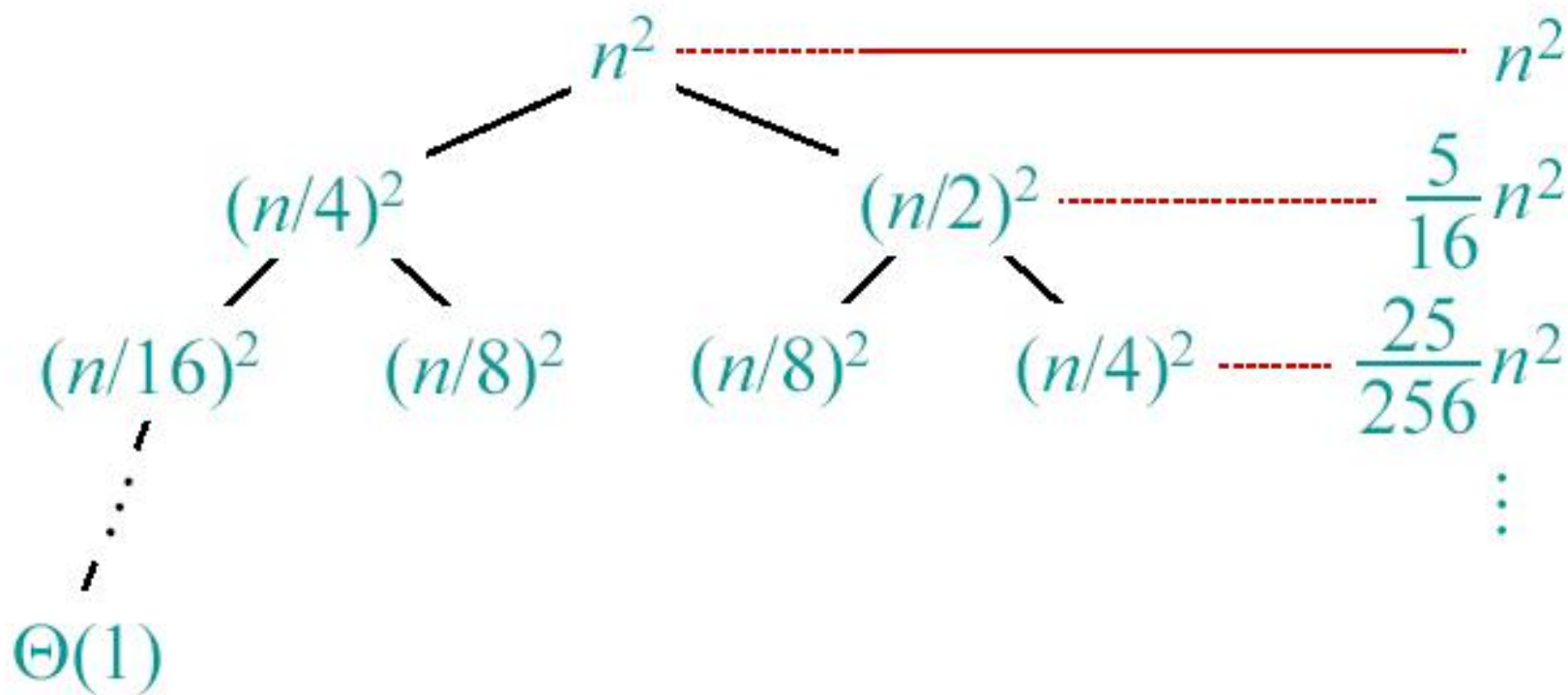
递归树方法举例

求解 $T(n) = T(n/4) + T(n/2) + n^2$:



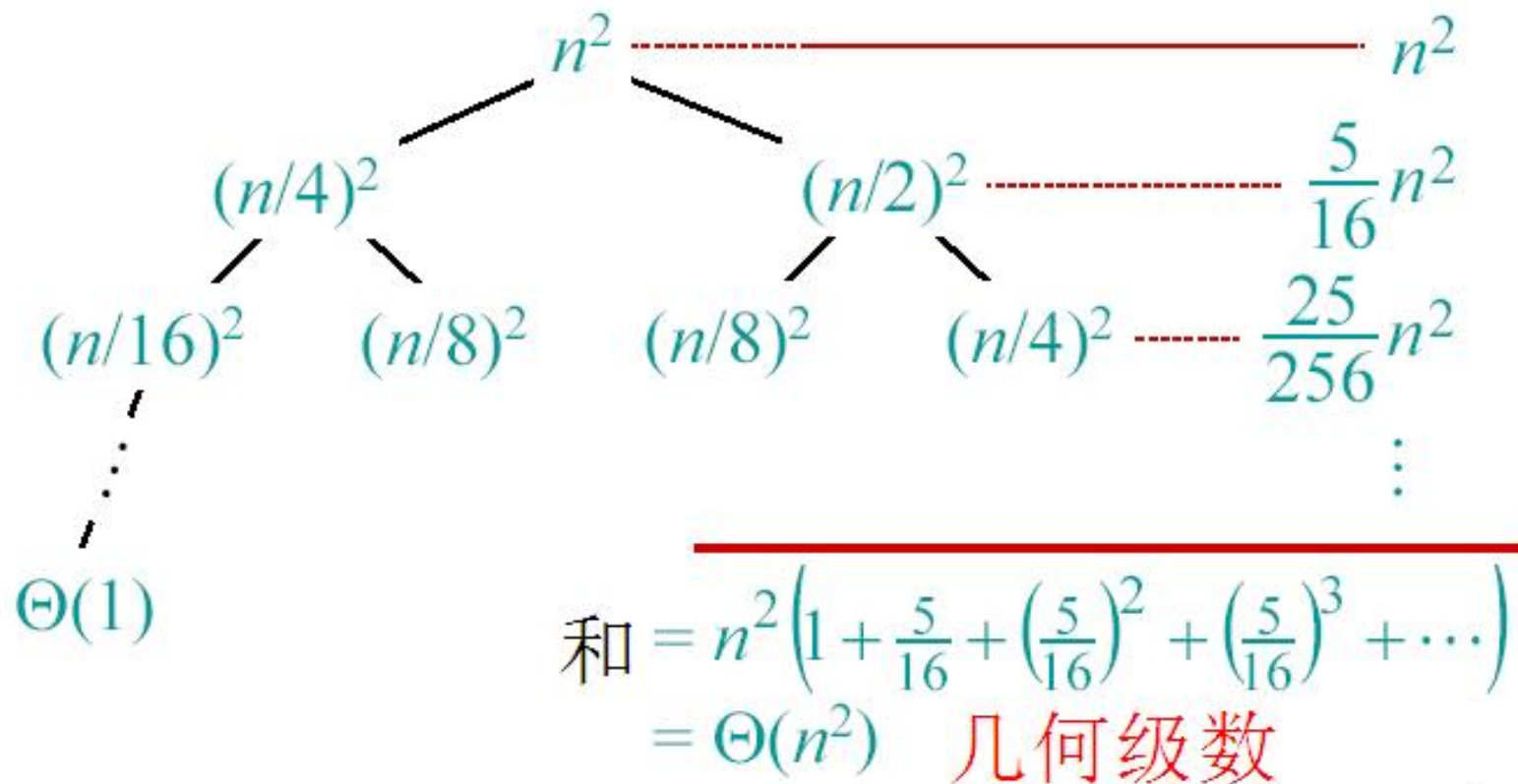
递归树方法举例

求解 $T(n) = T(n/4) + T(n/2) + n^2$:



递归树方法举例

求解 $T(n) = T(n/4) + T(n/2) + n^2$:



主方法适用于下面的递归形式

$$T(n) = aT(n/b) + f(n),$$

$a \geq 1, b > 1, f$ 为正的渐进函数

比较 $f(n)$ 和 $n^{\log_b a}$:

1. $f(n) = O(n^{\log_b a - \varepsilon}), \quad \varepsilon > 0$

$f(n)$ 比多项式 $n^{\log_b a}$ 增长的慢 (慢 n^ε 倍)

解: $T(n) = \Theta(n^{\log_b a})$.

2. $f(n) = \Theta(n^{\log_b a} \lg^k n)$, 常量 $k \geq 0$ 。

$f(n)$ 和 $n^{\log_b a} \lg^k n$ 增长速度相同

解: $T(n) = \Theta(n^{\log_b a} \lg^{k+1} n)$

三种情况（续）

比较 $f(n)$ 和 $n^{\log_b a}$ ：

3. $f(n) = \Omega(n^{\log_b a + \varepsilon})$, 常量 $\varepsilon > 0$

$f(n)$ 增长比多项式 $n^{\log_b a}$ 快（快 n^ε 倍）

并且 $f(n)$ 满足规定条件

$af(n/b) \leq cf(n)$, 常量 $c < 1$ 。

解： $T(n) = \Theta(f(n))$ 。

若 $f(n) = n^k$, 则比较 $\log b^a$ 和 k :

(1) 若 $\log b^a > k$, $T(n) = \Theta(n^{\log_b a})$

(2) 若 $\log b^a = k$, $T(n) = \Theta(f(n) \lg n)$

(3) 若 $\log b^a < k$ 并且 $f(n)$ 满足规定条件

$a f(n/b) \leq c f(n)$, 常量 $c < 1$ 。

解: $T(n) = \Theta(f(n))$ 。

Ex. $T(n) = 4T(n/2) + n$

Ex. $T(n) = 4T(n/2) + n$

$$a = 4, b = 2 \Rightarrow n^{\log_b a} = n^2; f(n) = n.$$

CASE 1: $f(n) = O(n^{2-\varepsilon})$ for $\varepsilon = 1$.

$$\therefore T(n) = \Theta(n^2).$$

Ex. $T(n) = 4T(n/2) + n^2$

$$a = 4, b = 2 \Rightarrow n^{\log_b a} = n^2; f(n) = n^2.$$

CASE 2: $f(n) = \Theta(n^2 \lg^0 n)$, that is, $k = 0$.

$$\therefore T(n) = \Theta(n^2 \lg n).$$

Ex. $T(n) = 4T(n/2) + n^3$



Ex. $T(n) = 4T(n/2) + n^3$

$$a = 4, b = 2 \Rightarrow n^{\log_b a} = n^2; f(n) = n^3.$$

CASE 3: $f(n) = \Omega(n^{2+\varepsilon})$ for $\varepsilon = 1$

and $4(cn/2)^3 \leq cn^3$ (reg. cond.) for $c = 1/2$.

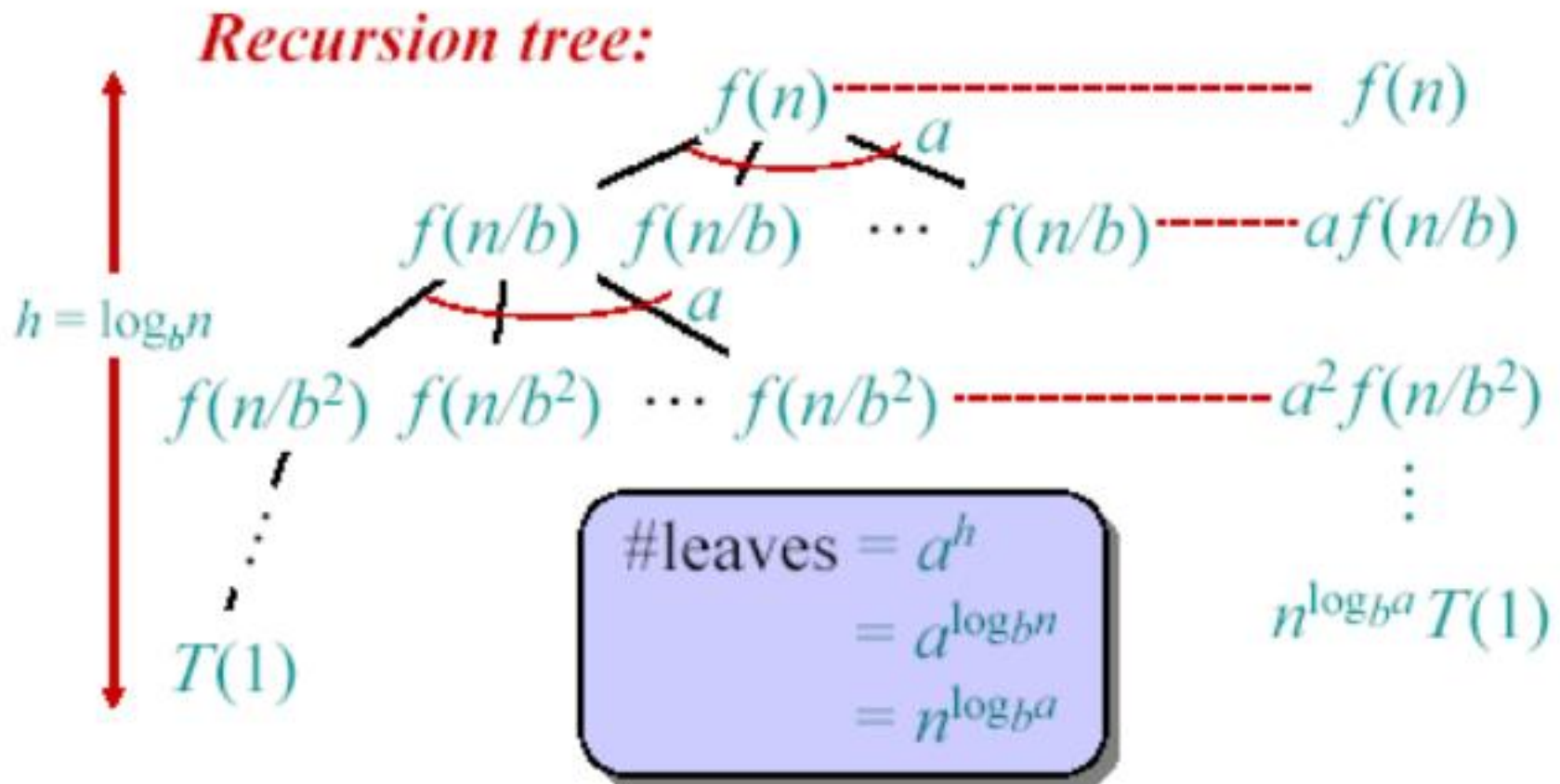
$$\therefore T(n) = \Theta(n^3).$$

Ex. $T(n) = 4T(n/2) + n^2/\lg n$

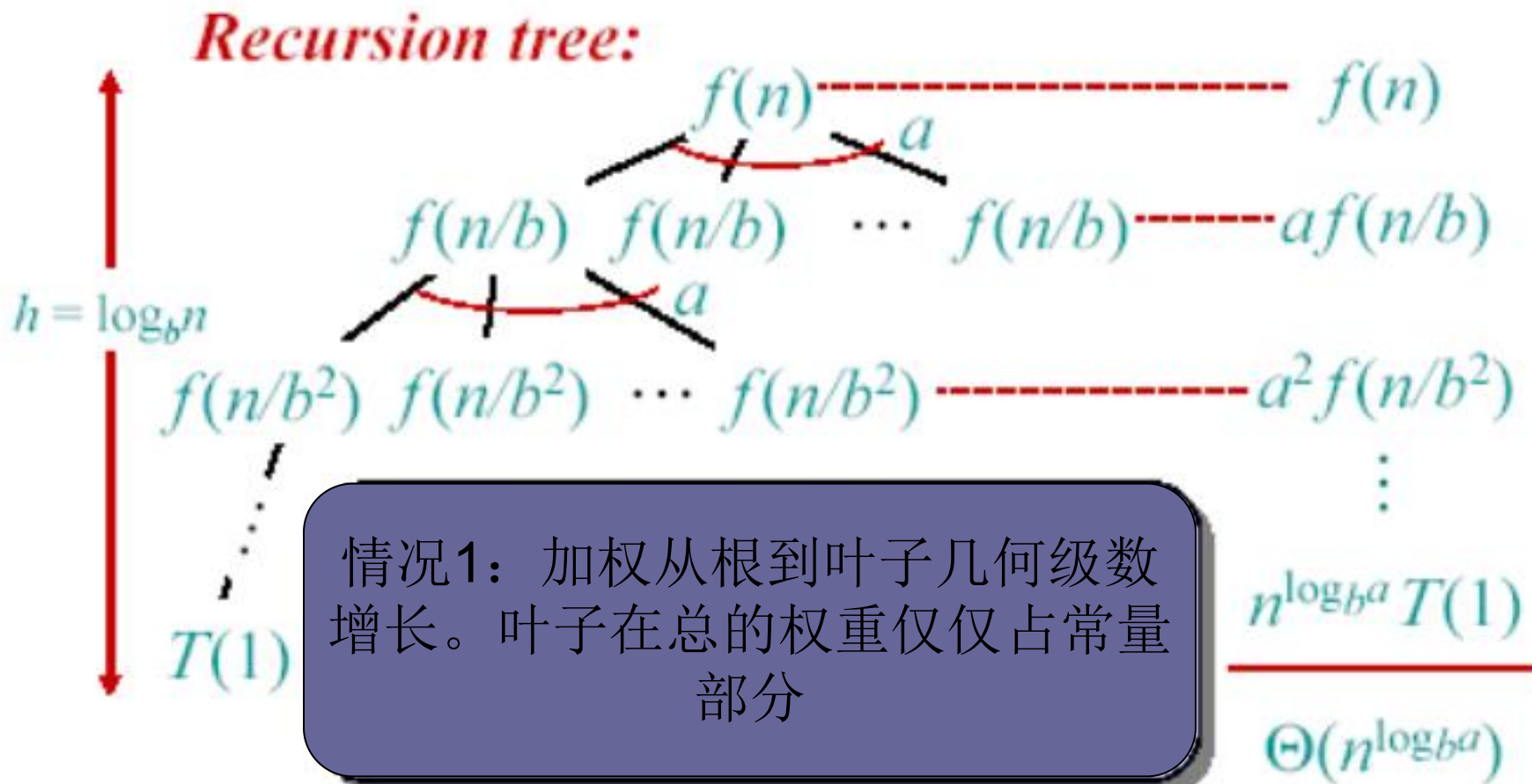
$$a = 4, b = 2 \Rightarrow n^{\log_b a} = n^2; f(n) = n^2/\lg n.$$

这时主方法不适用。

主方法的思路

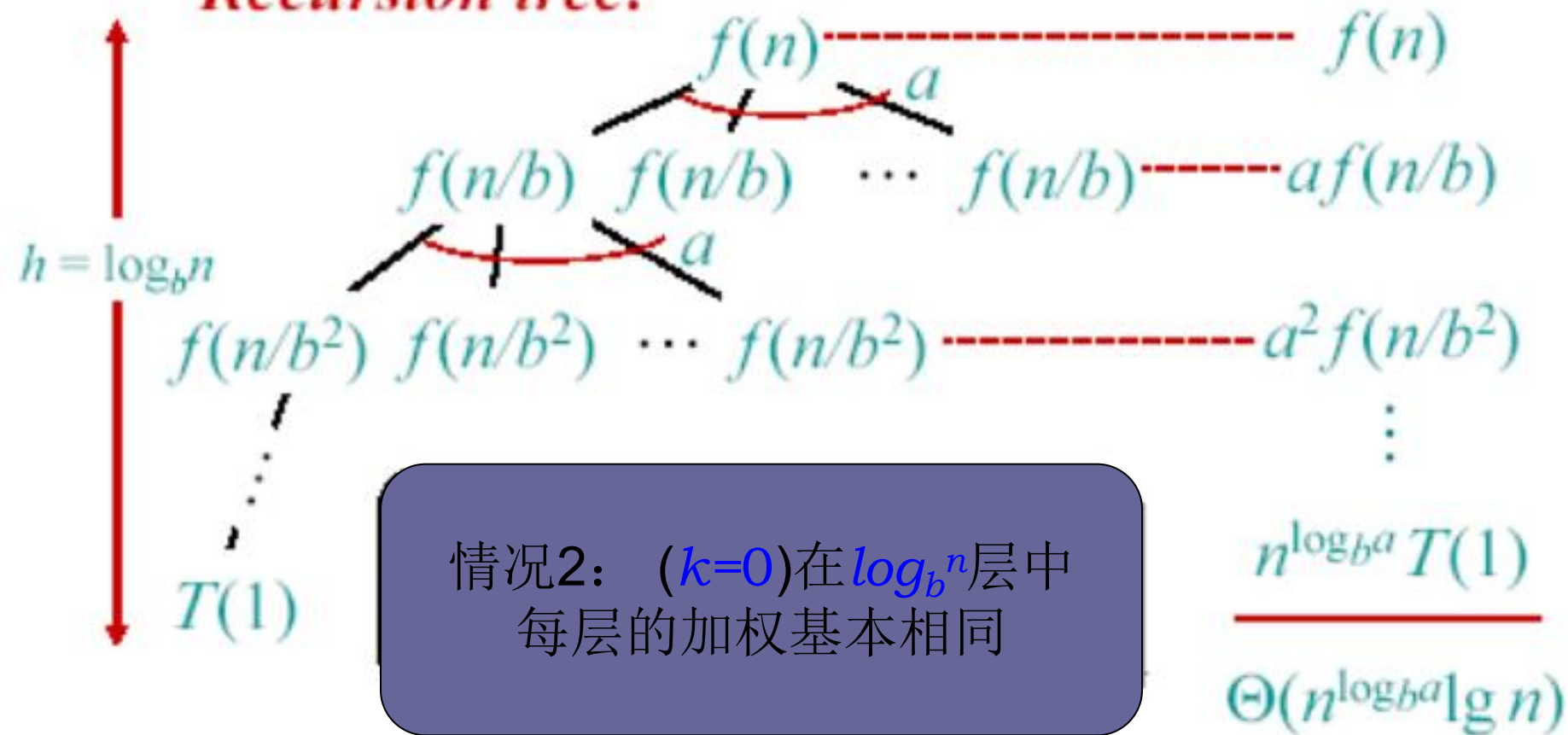


主方法的思路



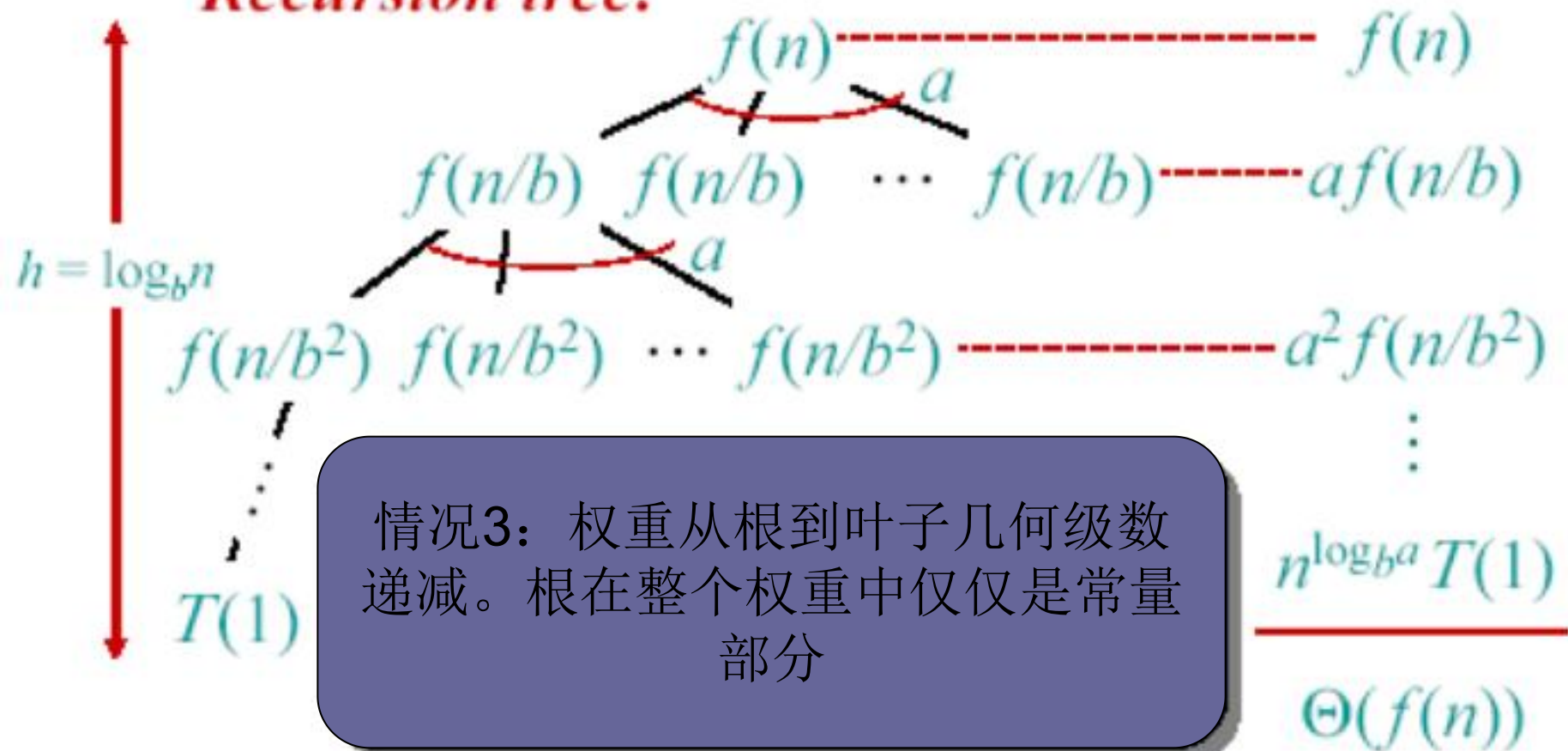
主方法的思路

Recursion tree:



主方法的思路

Recursion tree:



情况3：权重从根到叶子几何级数递减。根在整个权重中仅仅是常量部分



合肥工业大学
HEFEI UNIVERSITY OF TECHNOLOGY

Q/A?



合肥工业大学数据挖掘与智能计算“千人计划”研究团队
HFUT Data Mining and Intelligent Computing Laboratory